FGV EMAp

João Pedro Jerônimo

Aprendizado Profundo

Revisão para A2

Rio de Janeiro

2026

Conteúdo

1 Autoencoders 4
1.1 Introdução 5
1.2 Autoencoders 5
1.2.1 PCA (Principal Component Analysis) 6
1.2.2 AE (Autoencoder) 6
1.3 Autoencoders Convolucionais 6
1.4 Denoising Autoencoders 7
1.4.1 Speckel suppression in SAR Images 8
1.5 Variational Autoencoders 8
1.5.1 Treinamento 10
1.5.2 Geradores de Clima 12
1.5.3 Realismo 13
2 Aprendizado de Similaridade 14
2.1 Limitações de Classificadores 15
2.2 One-Shot Learning 15
2.3 Similarity Learning 16
2.4 Redes Siamesas 18
2.4.1 Constrastive Loss 18
2.4.2 Weighted Double Margin Contrastive (WDMC) Loss 19
2.4.3 Loss Tripla 19
2.5 Improving Similarity Learning 21
2.5.1 Melhorar a Amostragem 21
2.5.2 Melhorar a Loss 23
2.5.3 Ensemble Methods (Divide and Conquer) 24
3 Aprendendo em Dados Não-Rotulados 26
3.1 Motivação 27
3.2 Deep Clustering 27
3.2.1 Deep Embedding Clustering (DEC) 27
3.2.2 Deep K-Means 29
3.3 Self-Supervised Learning 30
3.4 Constrative Learning 31
3.4.1 Simple Framework for Contrastive Learning of Visual Representations (SimCLR) . 31
3.4.2 Aprendizado por Protótipos e Proporção de Rótulos (LLP) 33
3.5 Aprendizado com Rótulos Ruidosos 37
3.5.1 SELFIE 37
3.6 Aprendizado Fracamente Supervisionado 39
3.6.1 Class Activation Mapping (CAM) 39
3.7 Aprendizado Semi-Supervisionado 40
3.7.1 FixMatch 41
4 Self-Attention 43
4.1 Introdução 44
4.2 Self Attention para Processamento de Linguagem Natural 45
4.2.1 Mecanismo Inicial 45
4.2.2 Mecanismo Aprendível 46
4.2.3 Multihead Self Attention 47
4.3 Transformers 48

4.4 Transformers vão substituir RNNs? 49
4.5 Stand-Alone Self Attention 50
5 Visual Foundation Models (Visual FM) 52

Autoencoders

1.1 Introdução

No mundo real, os dados que recebemos são frequentemente incompletos, ruidosos ou de alta dimensionalidade. Então existe a necessidade de construir conhecimento a partir de dados imperfeitos. Um dos principais exemplos é a **falta de dados rotulados**. A maioria dos algoritmos de machine learning básicos necessitam que, pelo menos uma parte dos dados, possua rótulos e, a partir do padrão apresentado entre os dados rotulados, o modelo consegue **aprender**.

No âmbito específico de **modelos generativos**, como já comentamos na aula de GANs, partimos do pressuposto de uma **distribuição de probabilidade**, que gera os dados da vida real. No entanto, essa distribuição é desconhecida, complexa e de alta dimensionalidade. Para contornar esse problema, assumimos uma distribuição mais simples, chamada **distribuição latente**, que é mapeada para a distribuição de probabilidade dos dados reais com o intuito de aproximá-la. Então os modelos generativos tentam responder à pergunta:

Como podemos aprender uma p_{model} *similar à* p_{dados}

Uma grande vantagem dessa abordagem é que, como temos o conceito de distribuição, podemos ensinar o modelo **a partir dos outliers**. Mas por que isso seria interessante? A lógica vem de que, como outliers são casos muito raros e de difícil identificação, se o modelo consegue entender bem essas situações, os casos mais comuns são identificados com extrema facilidade. Em um conjunto de imagens de trânsito, a maioria das imagens seriam em climas ensolarados, um trânsito calmo e tranquilo, no entanto, alguns edge cases como esses:



Figura 1: Edge case de imagens de trânsito

1.2 Autoencoders

Um Autoencoder é uma rede neural feed-forward projetada para redução da dimensionalidade dos dados e, a partir das features aprendidas na dimensionalidade reduzida, reconstruir os dados originais.

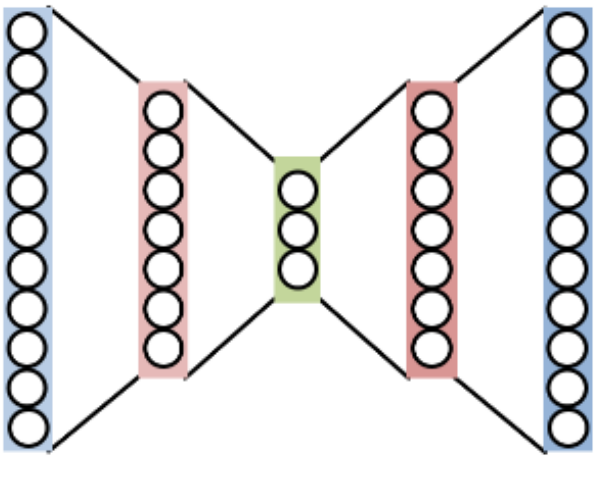


Figura 2: Desenho de esqueleto de Autoencoder

Antes de entrarmos nos conceitos mais modernos de Autoencoders, vamos ver alguns autoencoders que eram utilizados antes

1.2.1 PCA (Principal Component Analysis)

Não vamos entrar nos detalhes matemáticos do PCA, estes já foram abordados no *Resumo de Machine Learning A*. Mas intuitivamente, ele reduz a dimensionalidade dos dados através dos seus valores singulares, pegando as features **mais informativas**. Ou seja, ele faz uma **projeção linear** dos dados.

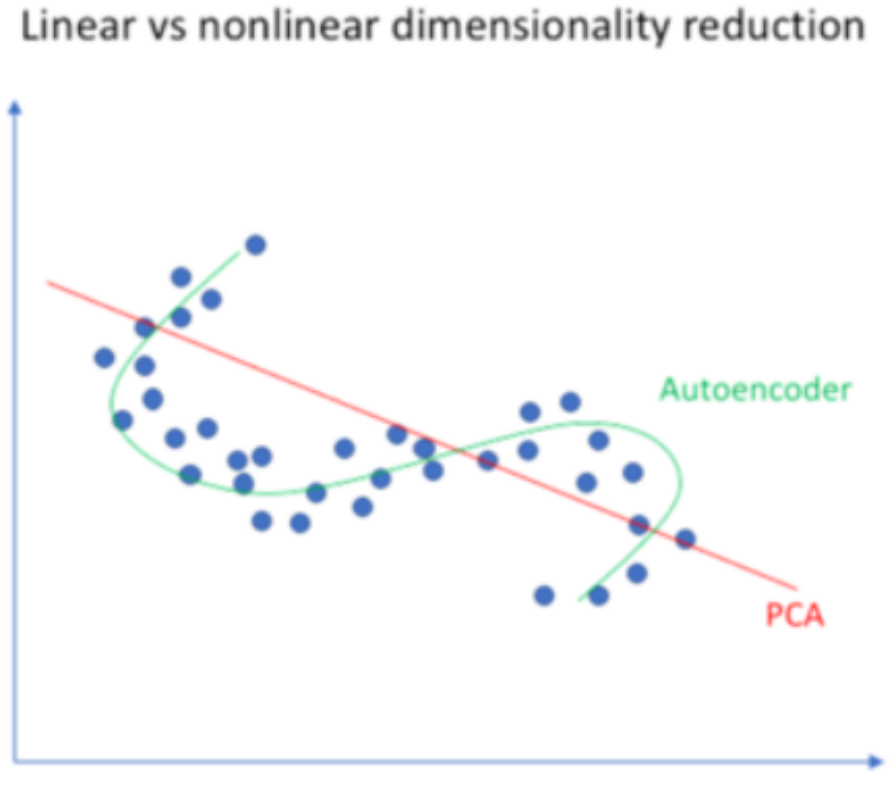


Figura 3: Exemplo de PCA em 2D para redução de dimensionalidade para 1D

1.2.2 AE (Autoencoder)

Já os autoencoders, tentam pegar as não-linearidades dos dados a partir das funções não-lineares das suas camadas, de forma que a loss que eles tentam otimizar foca em diminuir o quão distante os dados originais são dos gerados.

$$\mathcal{L}(W) = \frac{1}{N}\sum_{i}\|x_i - x_i'\|^2 + \lambda R(W) \tag{1}$$

A real é que não estamos interessados necessariamente nos dados gerados no output, mas sim no **bottleneck**, que é a camada de menor dimensionalidade no interior do autoencoder. Essa camada é a que contém as features mais informativas dos dados, e é a partir dela que podemos fazer **downstream tasks** como classificação, regressão, etc.

1.3 Autoencoders Convolucionais

Esses possuem um foco em imagens, onde as features são reduzidas através de camdas convolucionais. Onde são reduzidas e geradas novamente para se aproximar da imagem original.

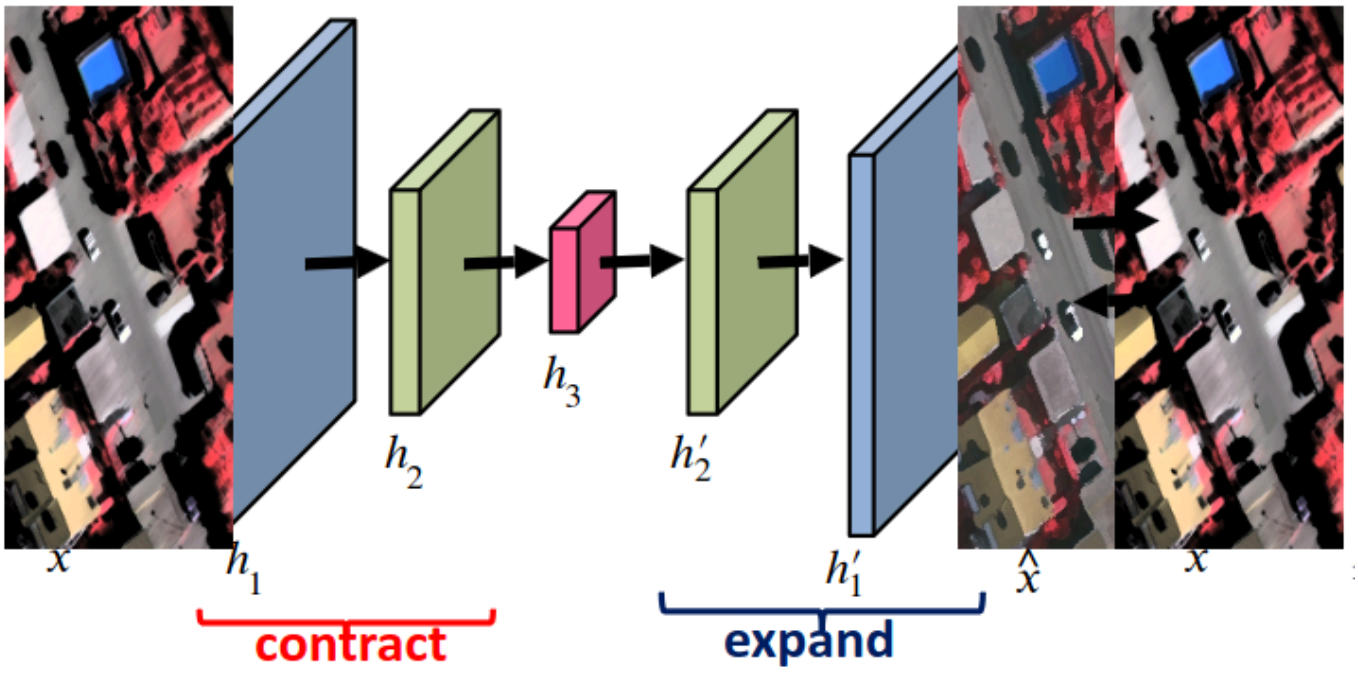


Figura 4: Exemplo de Autoencoder Convolucional

1.4 Denoising Autoencoders

Os denoising autoencoders são uma variação de autoencoders onde as imagens inseridas no input são **corrompidas** com algum tipo de ruído. A ideia por trás disso vem de que, se o modelo consegue aprender a reconstruir imagens ruidosas, imagens mais limpas são reconstruídas com mais facilidade. Não só isso, mas o encoder fica robusto à ruídos, o que é interessante para **downstream tasks** como classificação, onde imagens ruidosas podem ser classificadas com mais facilidade.

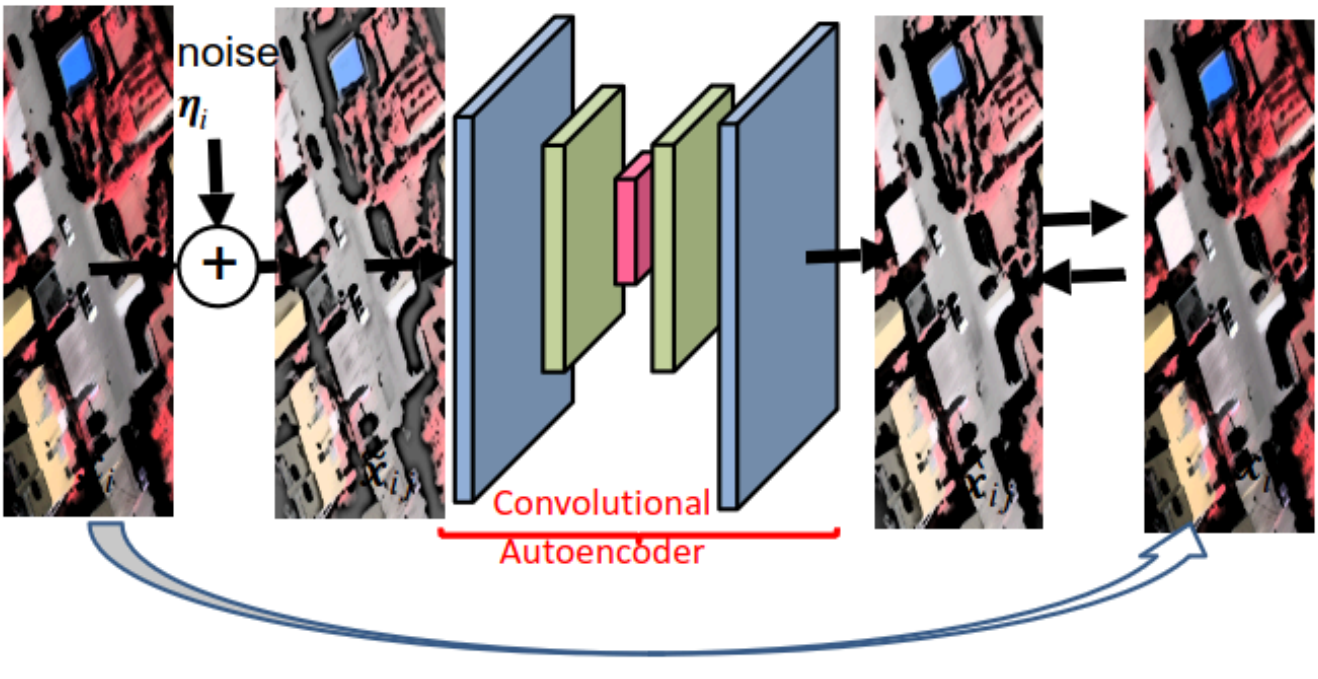


Figura 5: Exemplo de Denoising Autoencoder

Aqui, a loss é utilizada igual anteriormente, o ruído é aplicado apenas na entrada, assim o modelo não entende que o ruído é parte da imagem, mas sim que ele deve ser removido.

1.4.1 Speckel suppression in SAR Images

Os satélites de radar (SAR — **Synthetic Aperture Radar**) enviam pulsos de micro-ondas para a Terra e medem o sinal refletido. Devido ao fenômeno de interferência de ondas ao baterem no solo, a imagem gerada vem naturalmente com um ruído granulado chamado **speckle** (parece um "sal e pimenta" cobrindo a imagem).

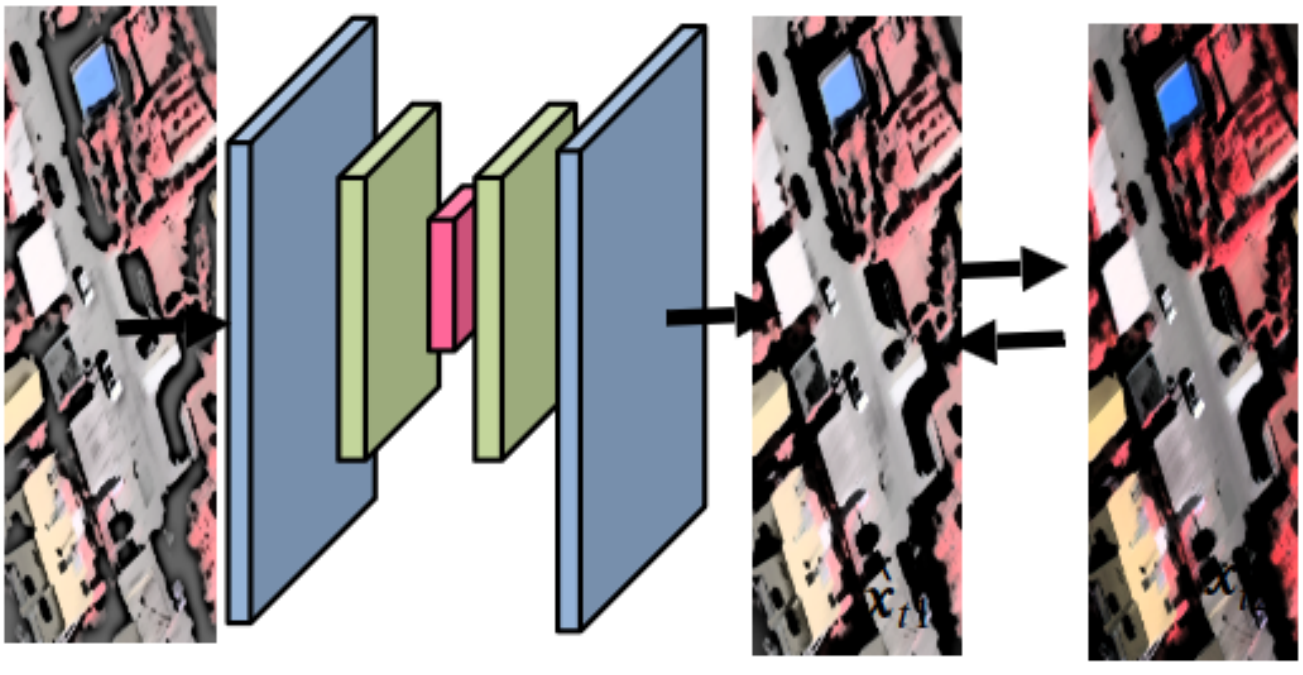


Figura 6: Exemplo de supressão de speckle em imagens SAR

No entanto, como falamos antes, o denoising autoencoder **ainda necessita de imagens limpas** para a sua loss, no entanto, com imagens de satélite, não temos imagens limpas. O que fazer então? Nós partimos de um pressuposto, se uma imagem foi tirada em um dia específico do ano, a imagem do mesmo local 1 dia depois (ou um período de tempo fixo determinado pelos cientistas) é **quase** a mesma imagem. Com essa premissa, assumimos que o ruído gerado em ambas as imagens é **aleatório** e **independente**, de tal forma que fazemos nossa rede receber a primeira imagem e tentar gerar a segunda. Isso remove o ruído justamente pela independência, fazendo com que a rede **não consiga aprender o padrão do ruído**, mas sim o padrão da imagem.

1.5 Variational Autoencoders

Se relembrarmos do que vimos em GANs, com os geradores, conseguimos gerar imagens a partir de uma distribuição latente.

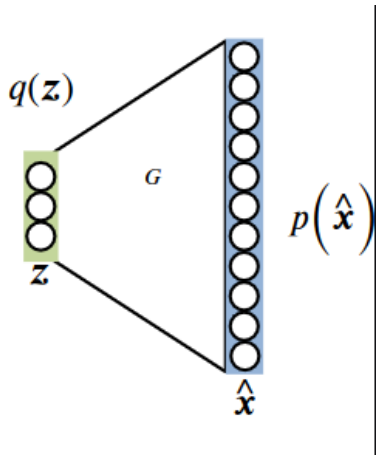


Figura 7: Exemplo de esqueleto de GAN

Nós temos a função $G : Z \rightarrow X$ que gera as minhas imagens. No entanto, não possuímos $G^{-1} : X \rightarrow Z$. E porque essa função poderia ser interessante? Se tivéssemos essa função, poderíamos modificar z para obter variações em características específicas da imagem/objeto desejado sujeitas à nossa vontade, por exemplo, mudar especificamente a cor dos olhos de uma pessoa.



Figura 8: Exemplo de variações de imagens geradas a partir de uma distribuição latente

Vamos assumir por um momento que a distribuição das imagens x advém de uma distribuição de uma variável latente z que **não conseguimos observar**. Em um esquema rápido, ficaria estruturado da seguinte forma:

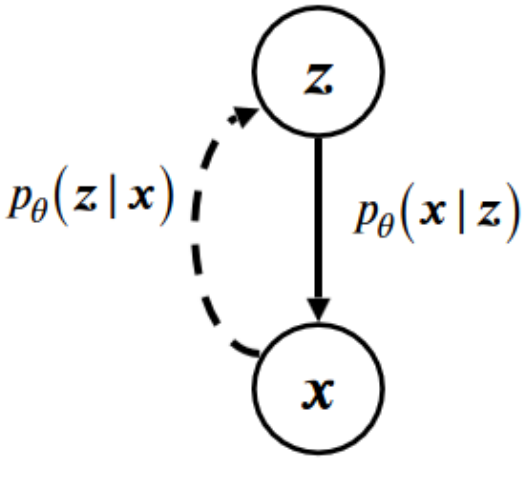


Figura 9: Exemplo de esqueleto de VAE

Nessa estrutura, algums métodos permitem estimar $p_\theta(x|z)$, no entanto, estimar $p_\theta(z|x)$ é **intratável**. A alternativa para isso, é utilizarmos uma distribuição **tratável** e **conhecida** $q_\varphi(z|x)$ e tentarmos, de alguma forma, aproximar $p_\theta(z|x)$ com $q_\varphi(z|x)$.

Na arquitetura dos Variational Autoencoders, em vez de tentar determinar features do espaço latente, tentamos associar cada entrada da distribuição com os parâmetros da distribuição conhecida $q_\varphi(z|x)$. De forma que depois do ajuste de φ, geramos novos valores para a variável latente z e, a partir da região do decoder (que representa $p_\theta(x|z)$), conseguimos gerar novas imagens.

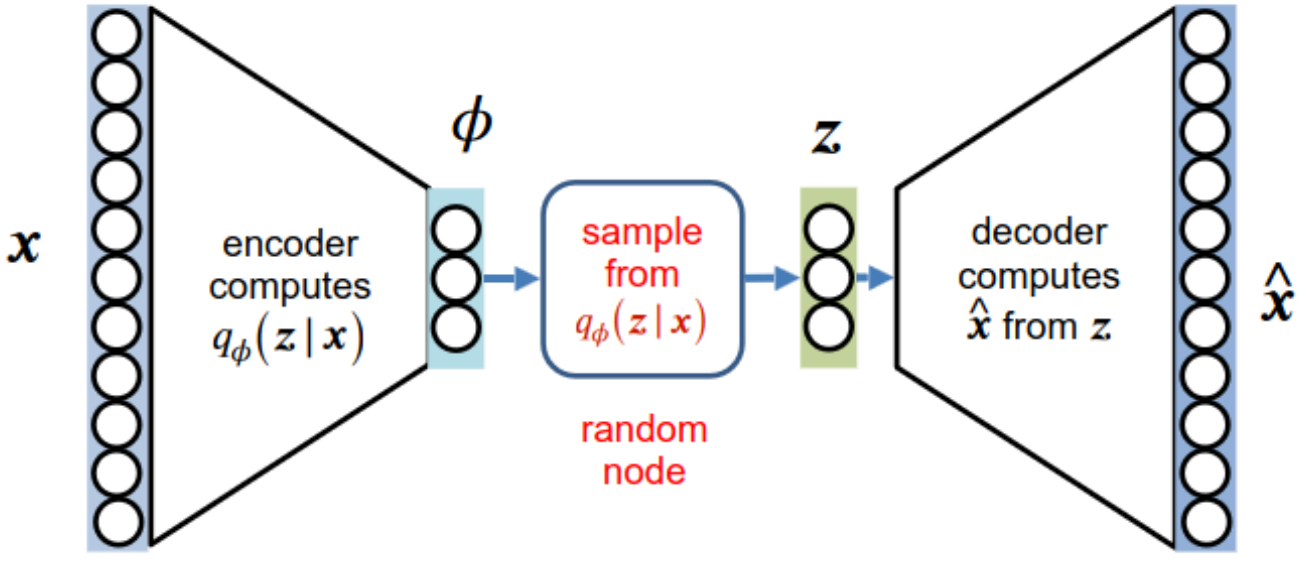


Figura 10: Exemplo de esqueleto de VAE

A loss dessa estrutura comprime dois termos principais

$$\mathcal{L}(\theta, \varphi, x) = \mathcal{L}_{\text{rec}}(\theta, \varphi, x) + \mathcal{L}_{\text{reg}}(\varphi) \tag{2}$$

o primeiro termo mede o quão bem o modelo consegue reconstruir a entrada, podemos usar, por exemplo:

$$\mathcal{L}_{\text{rec}}(\theta, \varphi, x) = \sum_i \|x_i - x_i'\|^2 \tag{3}$$

enquanto o termo de regularização mede o quão próximo a distribuição $q_\varphi(z|x)$ está da distribuição $p(z)$, que é a distribuição latente que assumimos. Para isso, podemos utilizar a divergência de Kullback-Leibler:

$$\mathcal{L}_{\text{reg}}(\varphi) = D_{\text{KL}}\big(q_\varphi(z|x) \parallel p(z)\big) \tag{4}$$

onde normalmente definimos $p(z)$ como uma distribuição normal multivariada padrão, ou seja, $p(z) = N(0, I)$. Essa regularização é importante pois o espaço latente deve ter duas características principais:

1. Continuidade: pequenas mudanças em z devem resultar em pequenas mudanças em x.
2. Completude: qualquer ponto no espaço latente deve gerar uma amostra válida e com significado.

$$D_{\text{KL}}\big(q_\varphi(z|x) \parallel p(z)\big) = \frac{1}{2}\sum_{j=1}^{J}\big(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\big) \tag{5}$$

Utilizar a distribuição normal encoraja o encoder a distribuir os pontos no centro do espaço latente e penaliza pontos que se formam em clusters longe do centro.

1.5.1 Treinamento

Agora o ponto principal, como fazemos para treinar um VAE? De início parece fácil. Temos nossa loss, basta fazer um backpropagation pelas suas camadas. No entanto, quando chegamos na camada de amostragem, temos um problema.

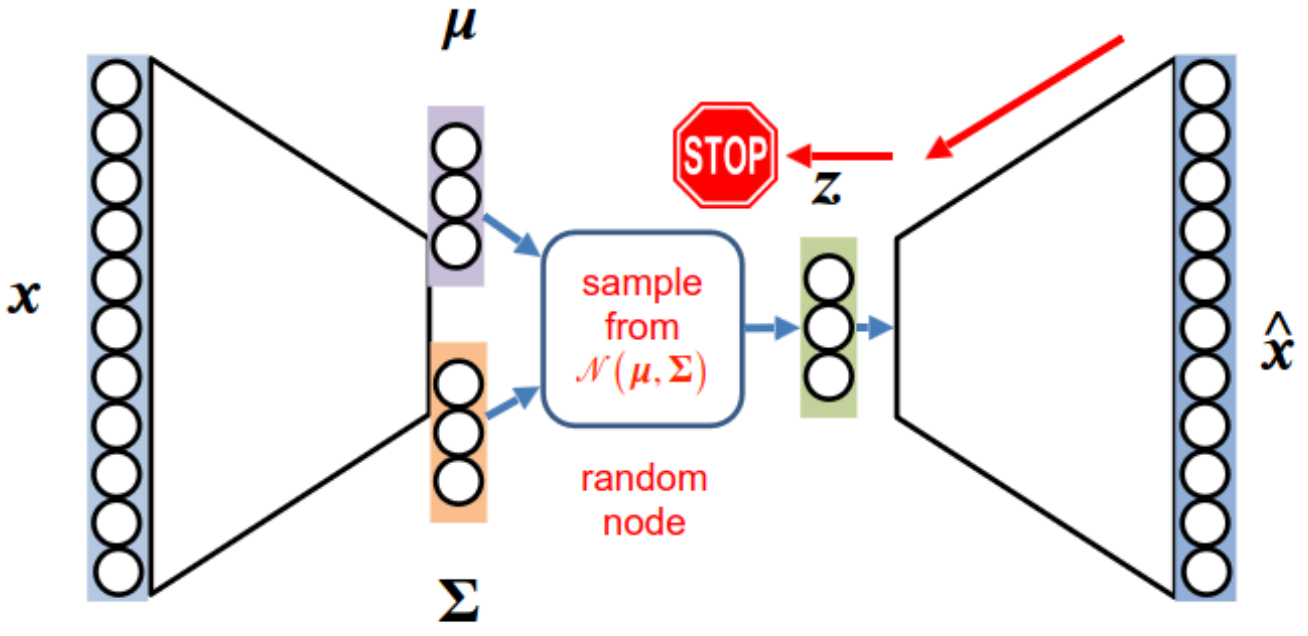


Figura 11: Ilustração do problema de backpropagation em VAE

O problema é que, como dito antes, nós precisamos amostrar da distribuição $q_\varphi(z|x)$, mas a amostragem não é diferenciável, então não conseguimos fazer backpropagation. A solução para isso é o **reparametrization trick**, onde ao invés de amostrar diretamente de $q_\varphi(z|x)$, nós amostramos de uma distribuição normal padrão e depois aplicamos uma transformação linear para obtermos a amostra desejada.

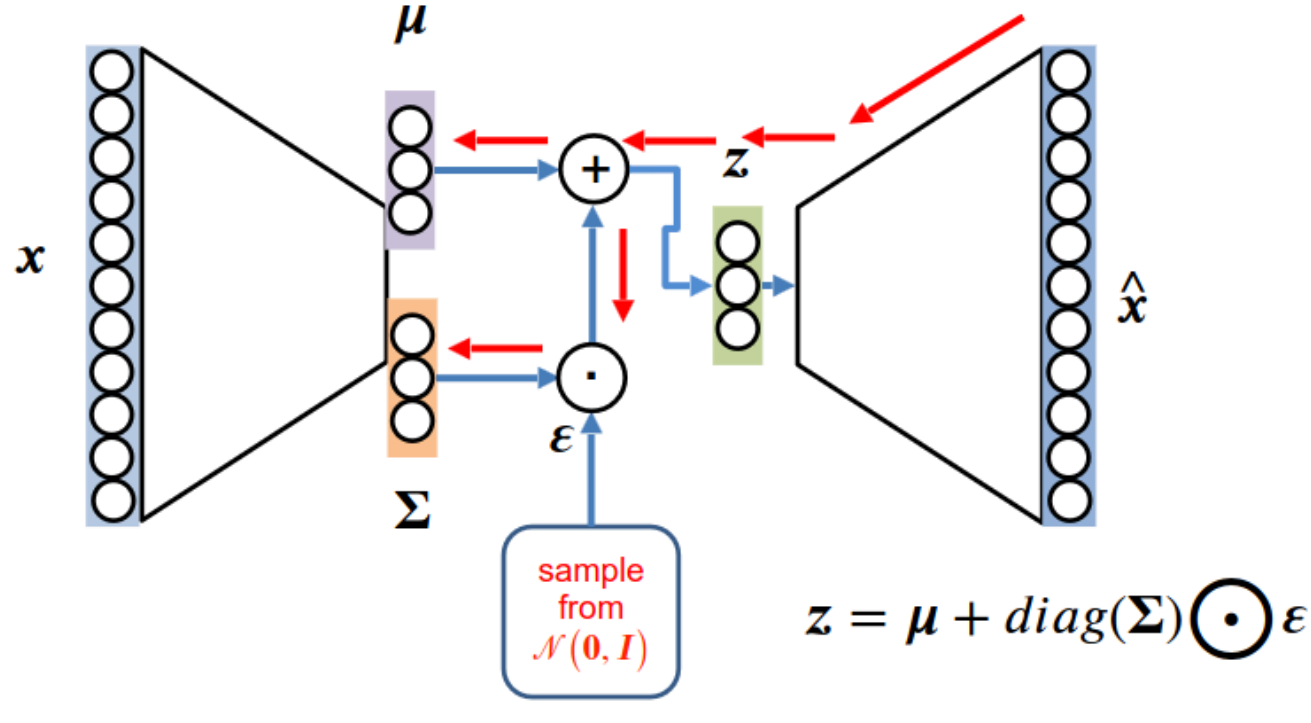


Figura 12: Exemplo de reparametrization trick

Acontece que, se amostrarmos $\varepsilon \sim N(0, 1)$, a variável latente z pode ser reparametrizada como:

$$z = \mu + \text{diag}(\Sigma) \cdot \varepsilon \tag{6}$$

de forma que isso é igual a amostrar z diretamente de uma $N(\mu, \Sigma)$ com a vantagem de podermos diferenciar com relação aos parâmetros.

1.5.2 Geradores de Clima

Uma aplicação interessante de VAE é a geração de imagens de clima. Queremos gerar **Weather Fields**, que é uma representação multidimensional (espaço + tempo) de variáveis meteorológicas, como vento, temperatura, pressão, etc. sobre uma região.

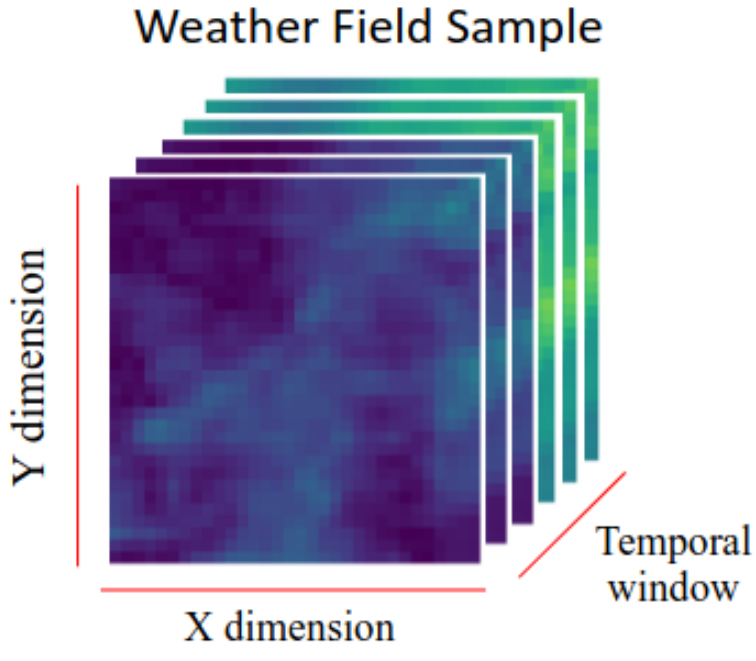


Figura 13: Exemplo de Weather Fields

Meu objetivo é, de alguma forma, ter um controle sobre o tipo de clima que eu gostaria de gerar em uma região. É aí que entra a sacada. Com os VAE, já que eu forço a distribuição latente a ser uma distribuição normal multivariada, o VAE consegue concentrar os climas comuns no centro do espaço latente e os climas extremos (e raros) nas caudas da distribuição.

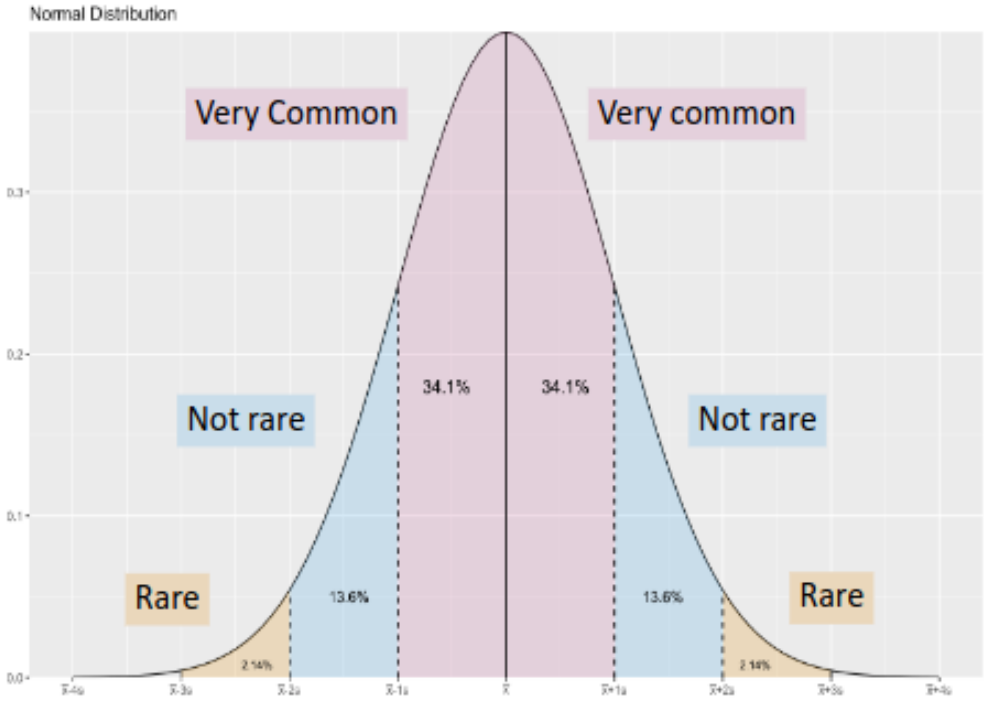


Figura 14: Exemplo de distribuição de climas no espaço latente

Dessa forma, se eu quiser simular eventos comuns naquela região, eu posso amostrar do centro da distribuição, enquanto se eu quiser simular eventos extremos, eu posso amostrar das caudas da distribuição. Isso é bom também pois não precisamos definir exatamente o que significa ser raro, o próprio VAE aprende isso a partir da distribuição latente.

Utilizando de gráficos QQ, os cientistas conseguiram mostrar que as imagens geradas pelo VAE dos eventos extremos se aproximavam muito com os dados reais observados.

1.5.3 Realismo

Para melhorar o realismo das imagens, conseguimos fazer uma mistura de conceitos que já vimos, usando GANs em conjunto com VAE. A ideia é que o GERADOR do GAN é o DECODER do VAE e o DISCRIMINADOR do GAN é um classificador que tenta diferenciar imagens reais de imagens geradas pelo DECODER. Dessa forma, o DECODER é forçado a gerar imagens mais realistas para enganar o DISCRIMINADOR.

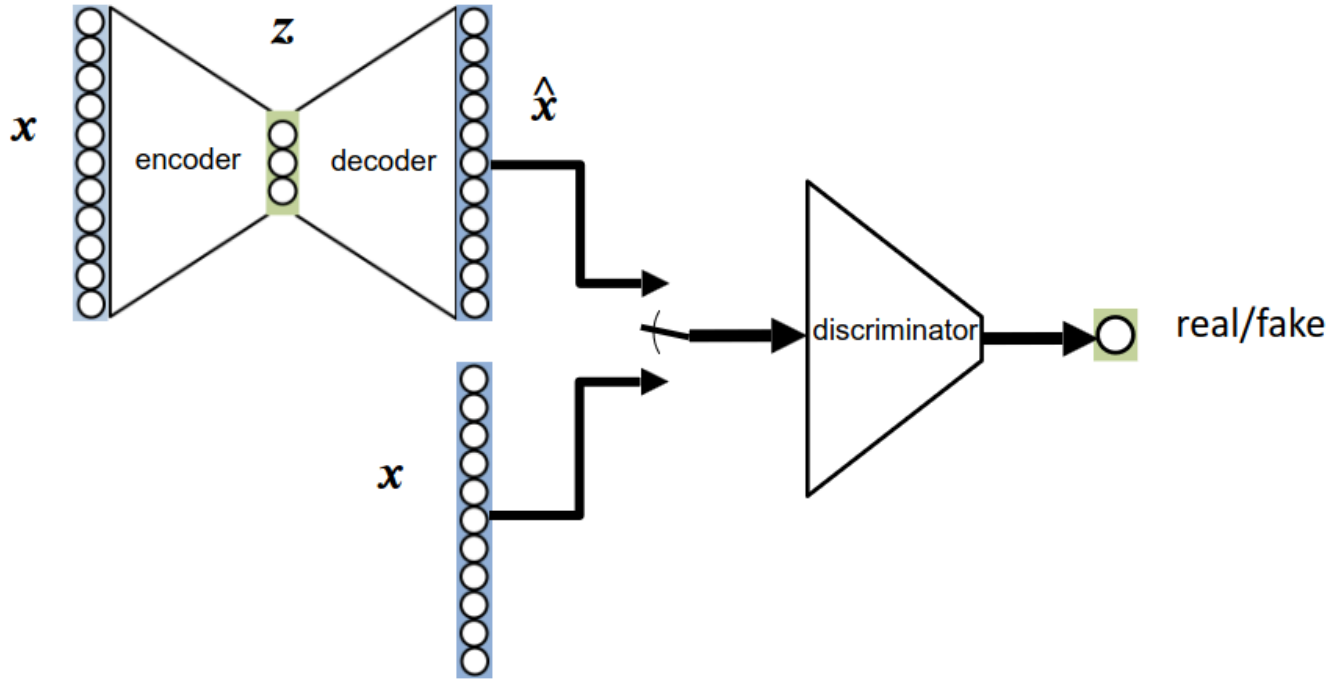


Figura 15: Exemplo de VAE-GAN

Aprendizado de Similaridade

2.1 Limitações de Classificadores

Tipicamente, quando a classificação envolve fitar o modelo em um conjunto de dados. No entanto, o modelo costuma pegar os padrões estruturais presentes naqueles dados. Por exemplo, se ele identifica imagens de animais, ele pode aprender que animais possuem olhos, patas, cauda, etc. No entanto, se ele for apresentado uma imagem do mesmo animal em posições diferentes, ele pode ter dificuldades de classificar corretamente.



Figura 16: Exemplo de limitação de classificadores

Outra limitação que podemos citar é o retreino desnecessário. Se temos um banco de dados e queremos adicionar novas imagens ou uma nova classe para enriquecer meu modelo, será necessário **treiná-lo do zero** novamente.

2.2 One-Shot Learning

Esse ramo de classificação tenta resolver o problema anterior. Nele, você tenta comparar duas imagens nunca vistas antes e determinar se elas são da mesma classe ou não. Ou seja, você não precisa fitar o modelo em um conjunto de dados, mas sim aprender a **comparar imagens**. Uma aplicação bem comum desse tipo de aprendizado é em Biometrias, como reconhecimento facial, onde você quer comparar a imagem de uma pessoa com a imagem que você tem no banco de dados para determinar se é a mesma pessoa ou não.

Uma forma mais simplificada utilizada em tarefas de visão computacional é tentar criar bijeções entre as imagens a partir de pontos específicos. A imagem a seguir exemplifica esse processo

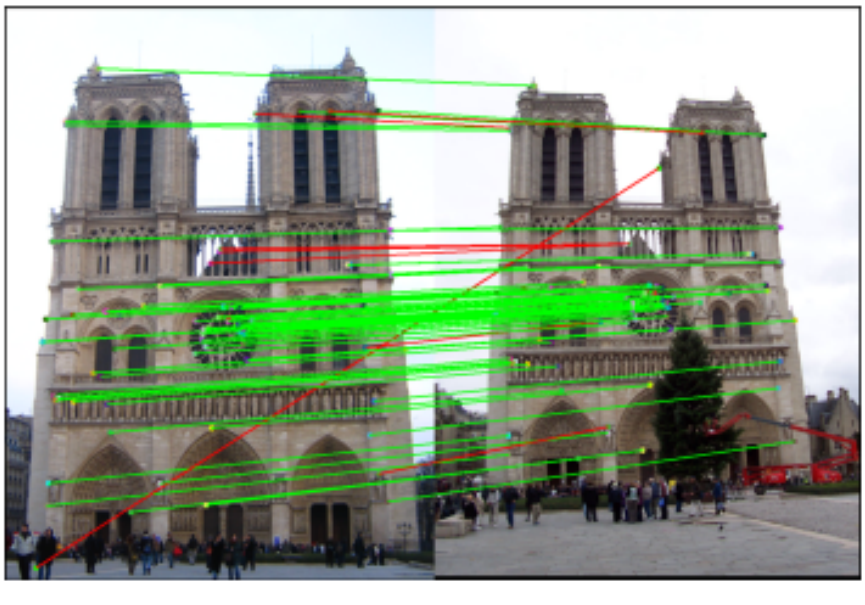


Figura 17: Exemplo de One-Shot Learning

Em diante, vamos utilizar o problema de **reconhecimento facial** como exemplo para explicar os conceitos do resto desse capítulo

2.3 Similarity Learning

Para contornar o problema da verificação, podemos trocar a estrutura do classificador! Podemos criar uma DNN que recebe duas imagens e retorna um valor entre 0 e 1, onde 0 significa que as imagens são diferentes e 1 significa que as imagens são iguais e utilizamos a loss da **Binary Cross Entropy** para treinar o modelo.

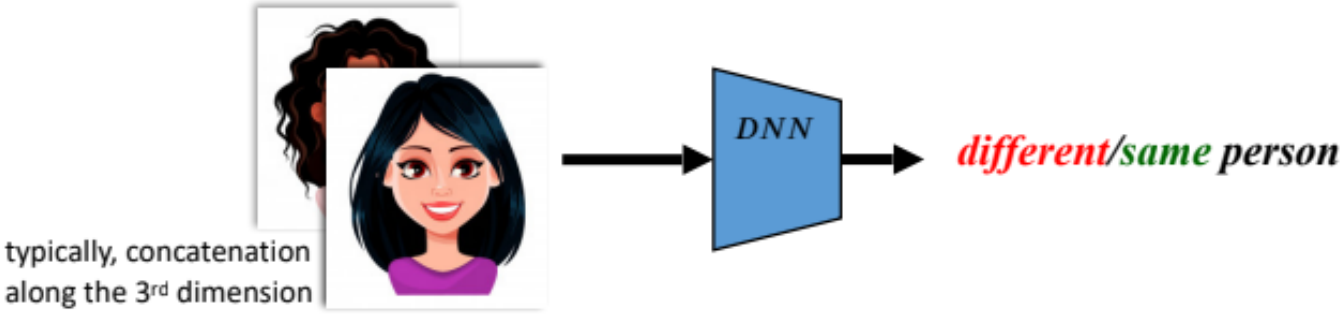


Figura 18: Exemplo de Similarity Learning

Mas essa abordagem não faz com que a rede aprenda a **medir o quão parecidas as imagens são**. Para esse objetivo, entramos em dois pontos que esses modelos de similaridade devem atingir:

1. **Aprender Métrica de Distância**: Dependendo do problema, precisamos de uma métrica de distância diferente. Por exemplo, para reconhecimento facial, a distância euclidiana é suficiente, mas para reconhecimento de imagens médicas, a distância de Mahalanobis pode ser mais interessante. Esse é um tópico de pesquisa ativo, onde o objetivo é aprender a métrica de distância mais adequada para o problema. O ponto é que precisamos de uma função que assinale pontuações altas/baixas para imagens parecidas/diferentes.
2. **Aprender Representações**: A rede precisa aprender a extrair features das imagens que sejam importantes e se encaixem nas características da métrica abordada. Isso é, atualmente, razoavelmente fácil de se fazer com redes profundas, pois elas próprias conseguem aprender as features importantes de acordo com a tarefa.

Podemos resumir o aprendizado de similaridade em uma frase: **Aprender um espaço de embeddings $\mathbb{R}^k$ onde imagens de pessoas parecidas estão próximas e imagens de pessoas diferentes estão distantes**.

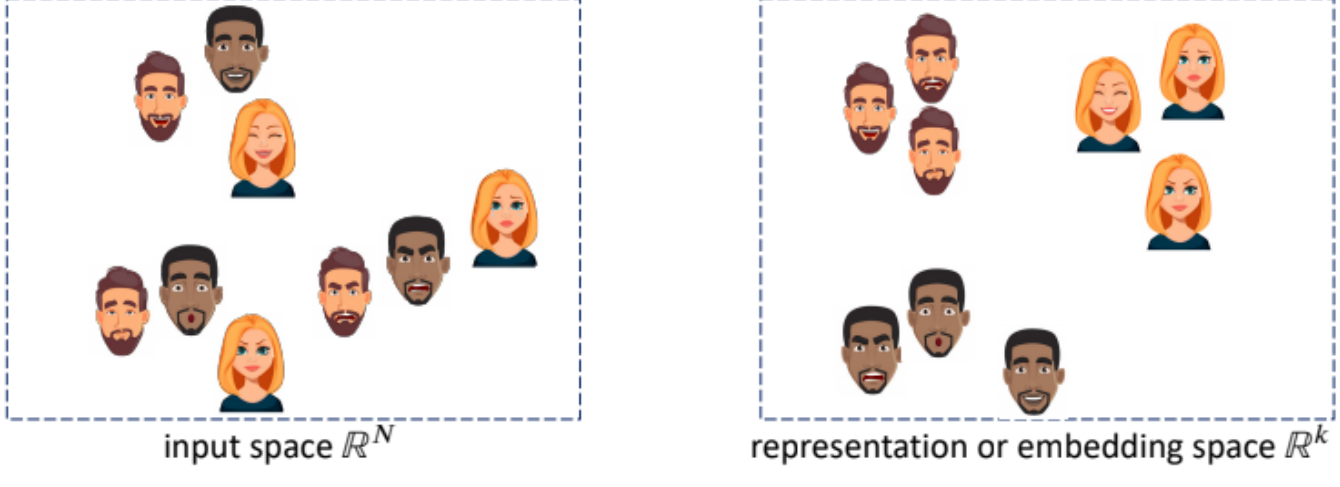


Figura 19: Exemplo de espaço de embeddings

Com isso em mente, podemos separar o processo de aprendizado em alguns passos

1. Precisamos encontrar um mapeamento $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ que mapeia as imagens de entrada do $\mathbb{R}^n$ para o espaço de embeddings $\mathbb{R}^k$. Precisamos aprender um mapeamento que satisfaça a característica de agrupamento citada anteriormente.

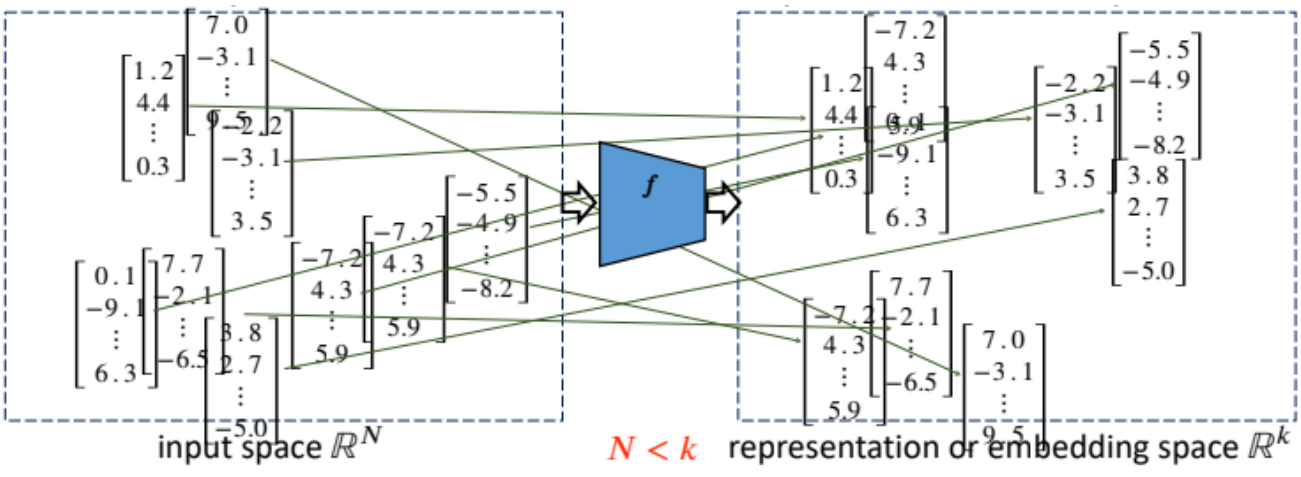


Figura 20: Exemplo de mapeamento para o espaço de embeddings

2. Agora é necessário escolher uma função de distância $d : \mathbb{R}^k x \mathbb{R}^k \rightarrow \mathbb{R}$ que mede a distância entre os embeddings. Essa função precisa ser escolhida de acordo com o problema, como citado anteriormente.

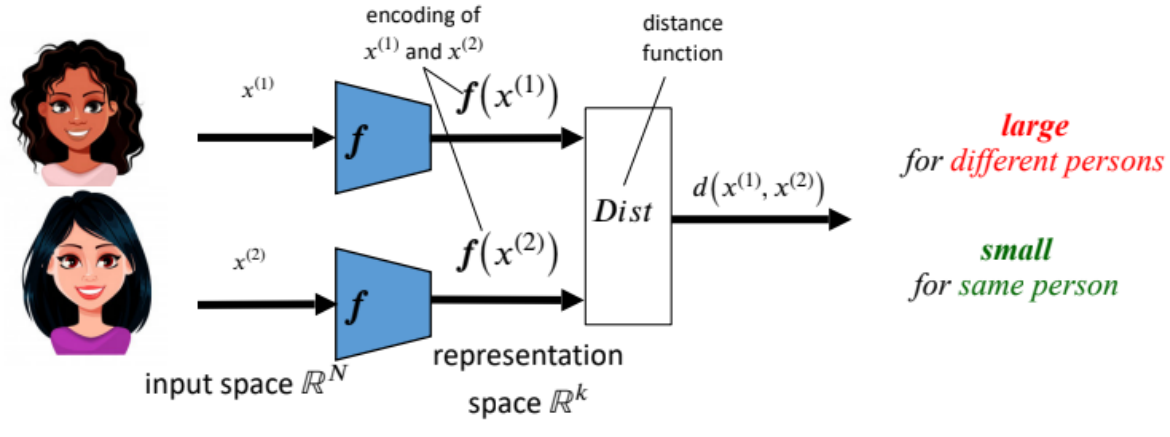


Figura 21: Exemplo de função de distância no espaço de embeddings

3. Agora precisamos definir um **threshold** τ que determina se duas imagens são da mesma classe ou não. Ou seja, se a distância entre os embeddings for menor que o threshold, as imagens são da mesma classe, caso contrário, são de classes diferentes.

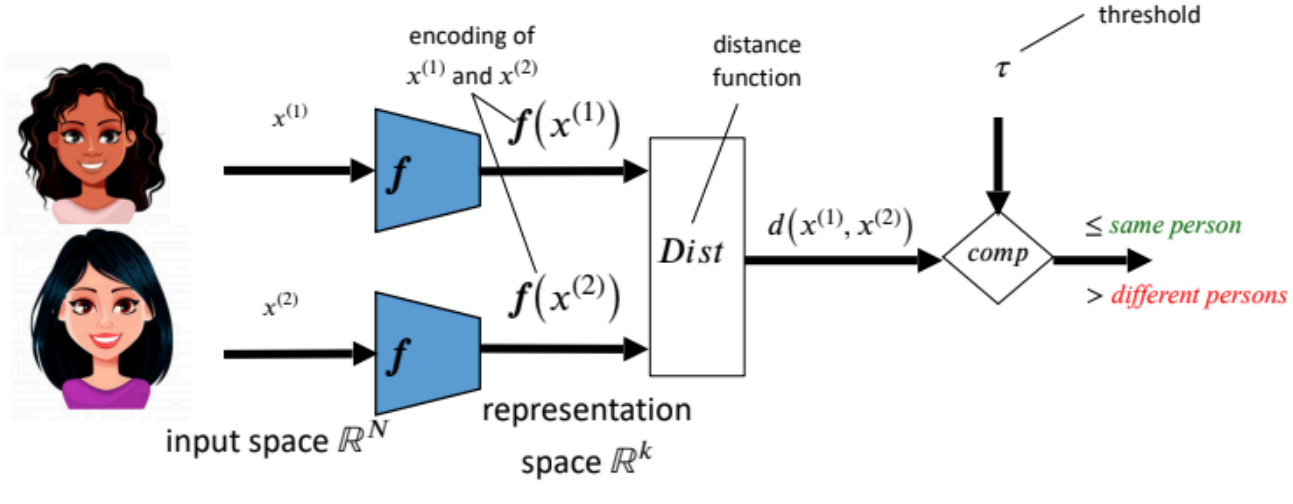


Figura 22: Exemplo de threshold no espaço de embeddings

2.4 Redes Siamesas

Redes siamesas é uma arquitetura de rede neural composta de duas redes **idênticas** que compartilham os mesmos pesos e parâmetros. O objetivo é que ambas as redes recebam uma imagem/entrada cada e realizem o mapeamento de $\mathbb{R}^n \rightarrow \mathbb{R}^k$. Depois que ambas fazem o mapeamento, tentamos medir o quão similares são os embeddings gerados. A ideia é que, se as imagens forem da mesma classe, os embeddings devem estar próximos, caso contrário, devem estar distantes.

$$d\left(x^{(1)}, x^{(2)}\right) = \left\| f\left(x^{(1)}\right) - f\left(x^{(2)}\right) \right\|^2$$

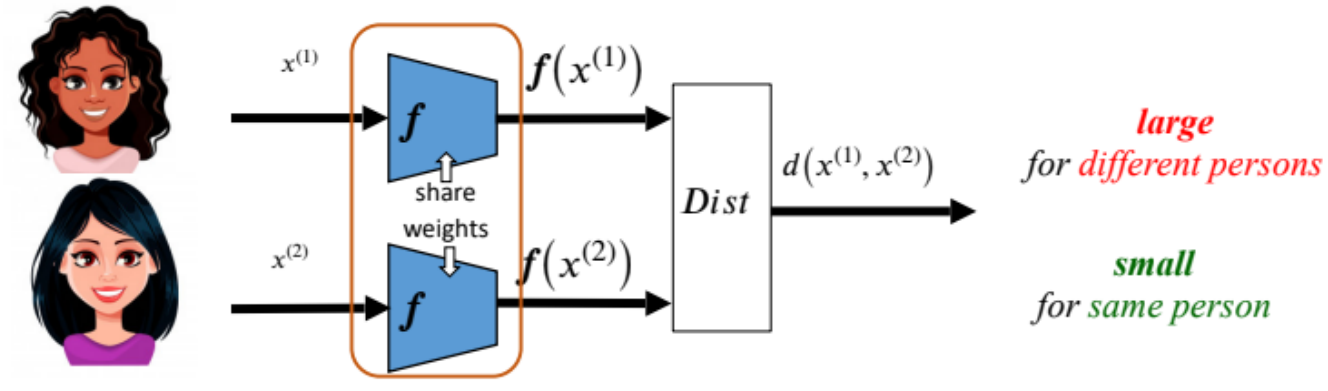


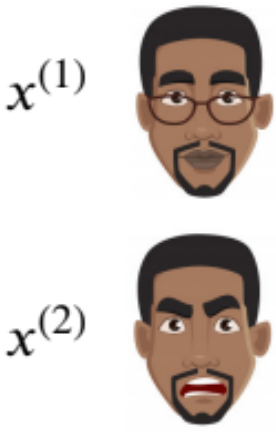
Figura 23: Exemplo de Rede Siamês

Cada imagem vai gerar um gradiente diferente nos parâmetros da rede. Para que ambas fiquem idênticas. Nós calculamos ambos os gradientes separadamente e depois fazemos uma média deles para atualizar os pesos da rede. Dessa forma, ambas as redes ficam idênticas.

2.4.1 Constrastive Loss

A loss utilizada para treinar redes siamesas é a **Contrastive Loss**. Vamos construí-la aos poucos.

Se A e B forem a mesma pessoa, gostaríamos que a loss gerasse um valor baixo, ou seja, que a distância entre os embeddings fosse pequena.



$$\mathscr{L}\left(x^{(1)}, x^{(2)}\right) = \left\| f\left(x^{(1)}\right) - f\left(x^{(2)}\right) \right\|^2$$

Figura 24: Exemplo de Contrastive Loss para imagens da mesma classe

Se A e B forem pessoas diferentes, gostaríamos que a loss gerasse um valor alto, ou seja, que a distância entre os embeddings fosse grande. No entanto, não é interessante que a loss seja muito grande, pois isso faria com que o modelo tentasse aprender a identificar características muito distintas. Intuitivamente, é como se ambas as imagens fossem tão diferentes, que não faz sentido o modelo tentar aprender todas as diferenças, não contribuem para o aprendizado. Então

definimos um **margin** m que determina o quão distante os embeddings devem estar. Ou seja, se a distância entre os embeddings for maior que o margin, a loss não precisa aumentar mais.

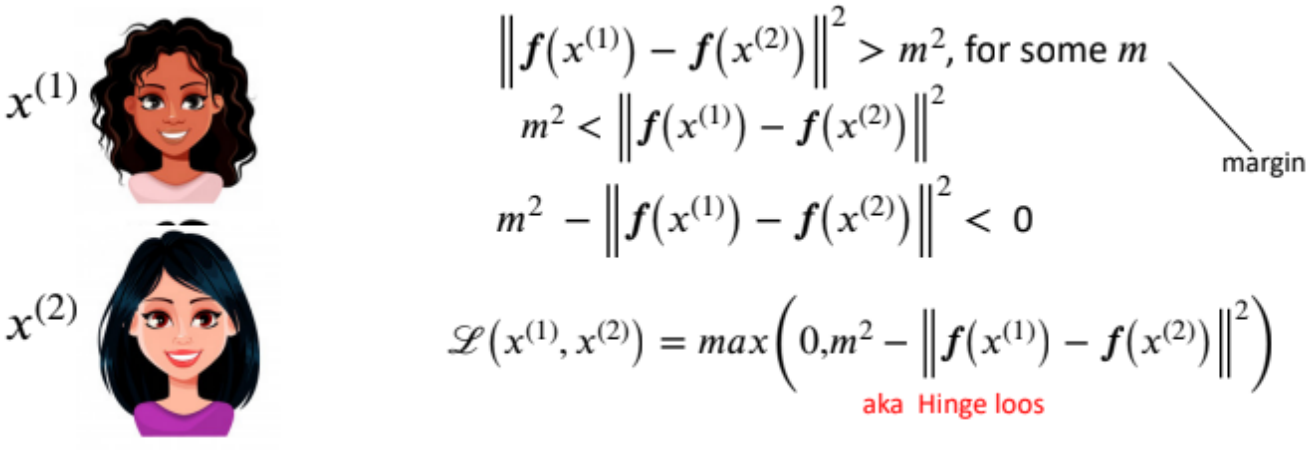


Figura 25: Exemplo de Contrastive Loss para imagens de classes diferentes

Então dado que t é uma variável indicadora que vale 1 se as imagens forem da mesma classe e 0 caso contrário, a loss final é definida como:

$$\mathcal{L} = t \cdot d^2 + (1 - t) \cdot \max(0, m^2 - d)^2 \tag{7}$$

onde d é a distância entre os embeddings e m é o margin. No problema de reconhecimento facial, d costuma ser a distância euclidiana.

2.4.2 Weighted Double Margin Contrastive (WDMC) Loss

Muito parecida com a contrastive loss. No entanto, adicionamos a lógica anterior de pares diferentes em pares iguais também. A lógica é que, se as duas imagens forem **muito** parecidas, não faz sentido fazer com que o modelo perca tempo aprendendo diferenças mínimas. Então definimos duas margens, m e M, uma para pares diferentes e outra para pares iguais.

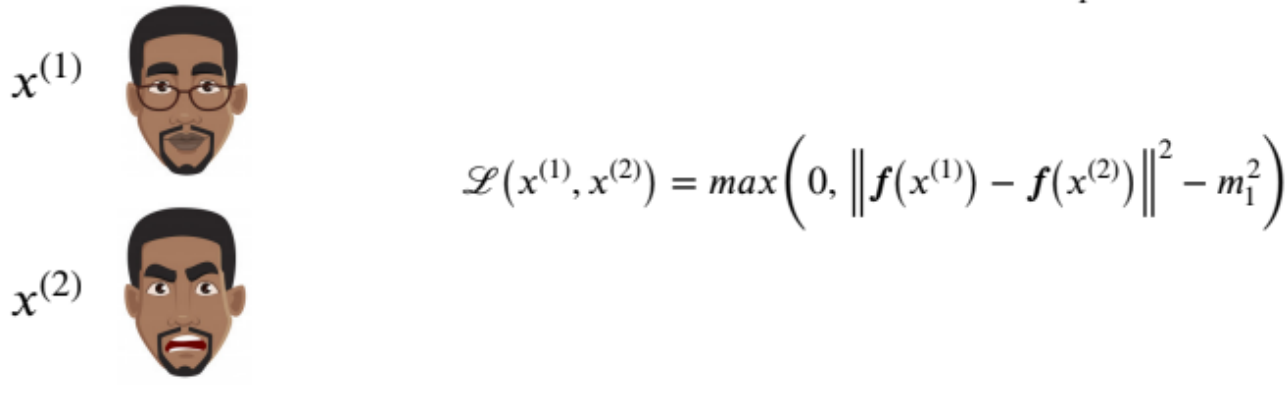


Figura 26: Exemplo de WDMC Loss

Dessa forma, a loss final é definida, para um batch $\left\{x_i^{(1)}, x_i^{(2)}\right\}_{i=1}^{N}$, como:

$$\mathcal{L} = \sum_i w_1 t_i \max\left(0, d\left(x_i^{(1)}, x_i^{(2)}\right) - m^2\right) + w_2(1 - t_i) \max\left(0, M^2 - d\left(x_i^{(1)}, x_i^{(2)}\right)\right) \tag{8}$$

2.4.3 Loss Tripla

Em vez de treinarmos em pares de imagens, podemos treinar em **trios** de imagens. Temos uma imagem de referência (Anchor A), uma imagem da mesma classe (Positive P) e uma imagem de

classe diferente (Negative N). A ideia é que a distância entre o Anchor e o Positive seja menor que a distância entre o Anchor e o Negative. Ou seja, queremos que:

$$d(A, P) < d(A, N) \tag{9}$$

logo, deixando mais restrito para uma margem m, queremos

$$d(A, P) - d(A, N) < -m \tag{10}$$

então temos que

$$\mathcal{L} = \max(0, d(A, P) - d(A, N) + m) \tag{11}$$

Para um batch $\{A_i, P_i, N_i\}_{i=1}^{N}$, a loss final é definida como:

$$\mathcal{L} = \sum_{i} \max(0, d(A_i, P_i) - d(A_i, N_i) + m) \tag{12}$$

A arquitetura vira um conjunto de 3 redes siamesas, uma para cada imagem, que compartilham os mesmos pesos e parâmetros. A loss é calculada a partir das distâncias entre os embeddings gerados pelas redes.

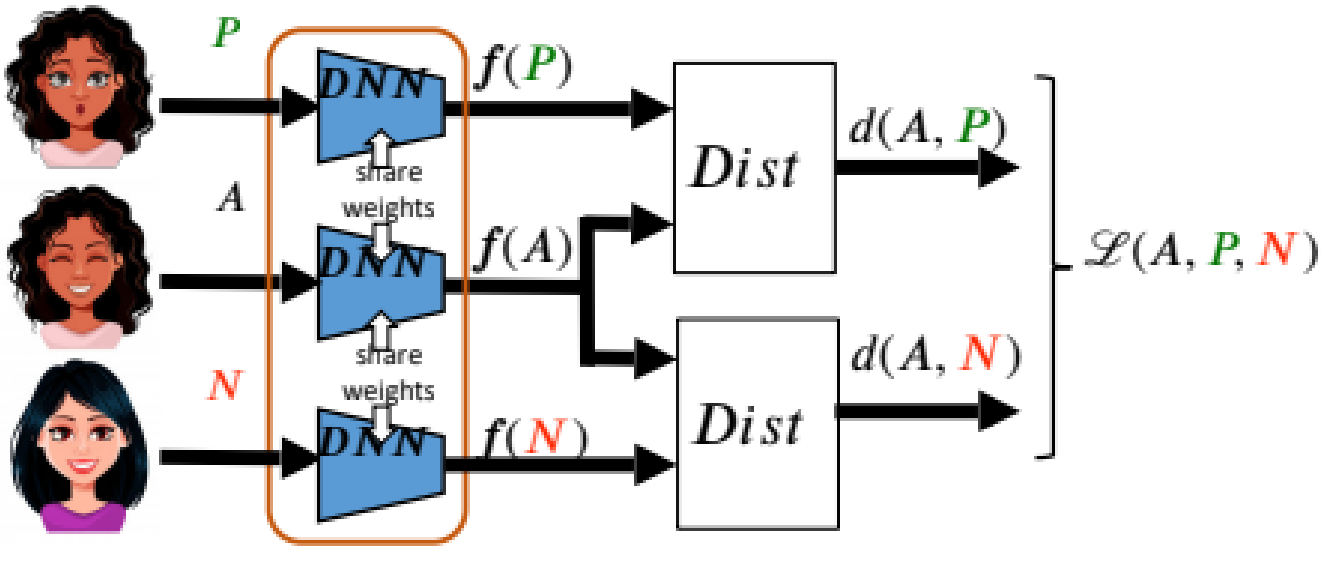


Figura 27: Exemplo de arquitetura para a Triplet Loss

E para a fase de **testes**, utilizamos a estrutura com apenas duas redes siamesas, uma para a imagem de referência e outra para a imagem a ser comparada.

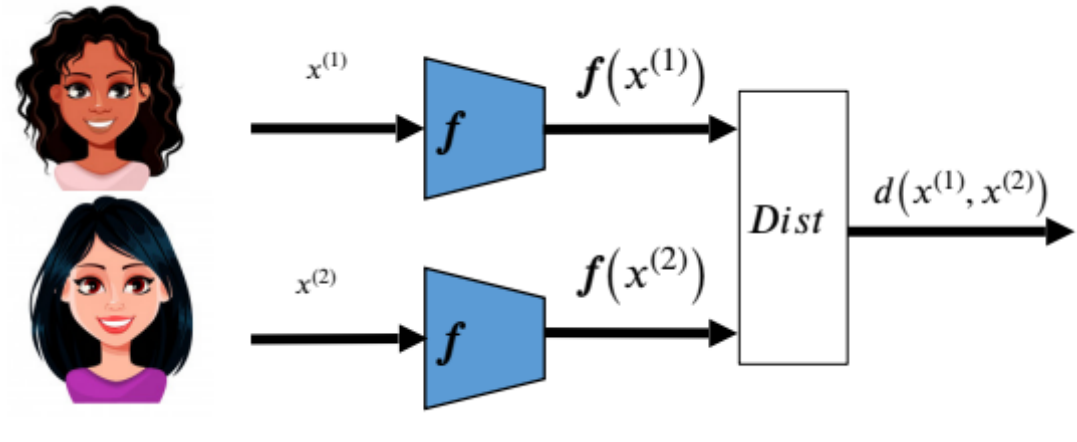


Figura 28: Exemplo de arquitetura para a Triplet Loss na fase de teste

2.5 Improving Similarity Learning

Existem alguma forma de melhorar o aprendizado por similaridade. Alguns pontos que vamos abordar são:

1. **Melhorar a Loss**
2. **Melhorar a Amostragem**
3. **Ensenble Methods**

A parte de melhoria da loss nós já comentamos com as abordagens de diferentes loss anteriormente (mesmo que ainda existem falhas que vão ser apontadas)! Então vamos focar nos outros pontos.

2.5.1 Melhorar a Amostragem

Vamos nesse momento, dar um pouco de foco na forma de abordagem usando **triplas** de imagens. O problema de trabalhar com triplas, é que o número total de combinações possíveis em um conjunto com n dados é da ordem de n^3. Ou seja, tanto você sintetizar todos esses trios, quanto você **treinar** com eles vai demorar bastante. Ainda existe o fato que, utilizando da triplet loss, **nem todos eles são úteis para o aprendizado**. Lembrando que para uma tripla específica:

$$\mathcal{L}(A, P, N) = \max(0, d(A, P) - d(A, N) + m) \tag{13}$$

então o gradiente dessa função fica

$$\frac{\mathrm{d}\max}{\mathrm{d}x} = \begin{cases} 0 & x \leq 0 \\ 1 & x > 0 \end{cases} \tag{14}$$

ou seja, para triplas onde a seguinte condição é verdade:

$$d(A, P) - d(A, N) + m \leq 0 \tag{15}$$

não vão contribuir para o aprendizado, pois o gradiente é zero!

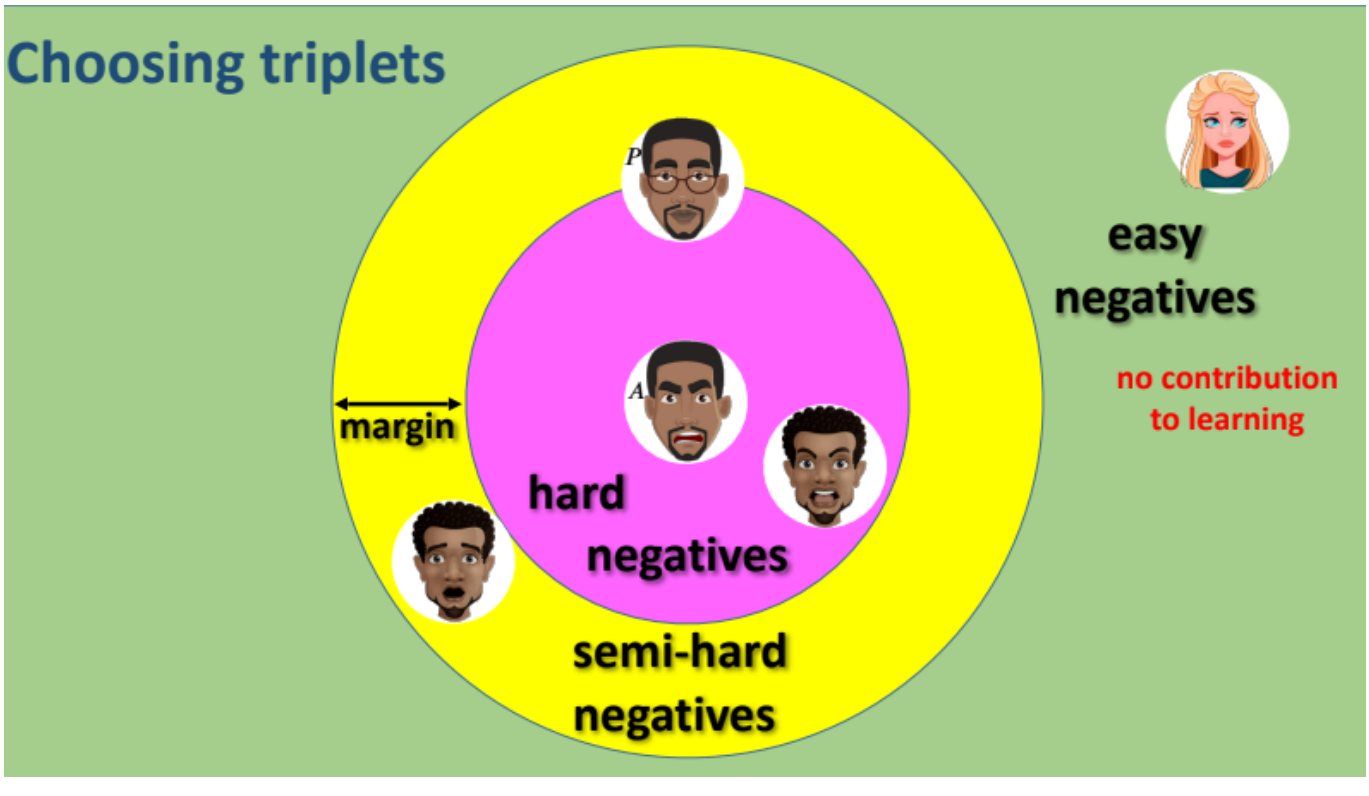


Figura 29: Exemplo de gradiente da Triplet Loss

Pela ilustração da imagem, queremos pegar exemplos **difíceis** ou **semi-difíceis**, pois eles geram os gradientes mais fortes para as losses. Um exemplo ingênuo de seleção de triplas, seria algo como:

```
selecionar triplas iniciais {A_i, P_i, N_i}
enquanto condição de parada = false {
    Treina por algumas épocas
    Escolhe triplas difíceis {A_j, P_j, N_j}
}
```

As triplas podem ser selecionadas de forma offline (manualmente) ou online (automaticamente). Vamos tentar pensar em algum método pra seleção de triplas online.

A ideia é termos uma **árvore** de classes. Essa árvore vai servir como um **GPS** do dataset, agrupando as imagens por similaridade visual. Em vez de testar tudo contra tudo, a rede vai consultar essa árvore pra responder duas perguntas:

1. *Como garanto que o batch tenha classes variadas e não fique viciado em poucas classes repetidas?*
2. *Para cada imagem de **âncora**, onde estão os negativos que são difíceis de verdade, mas sem pegar ruído?*

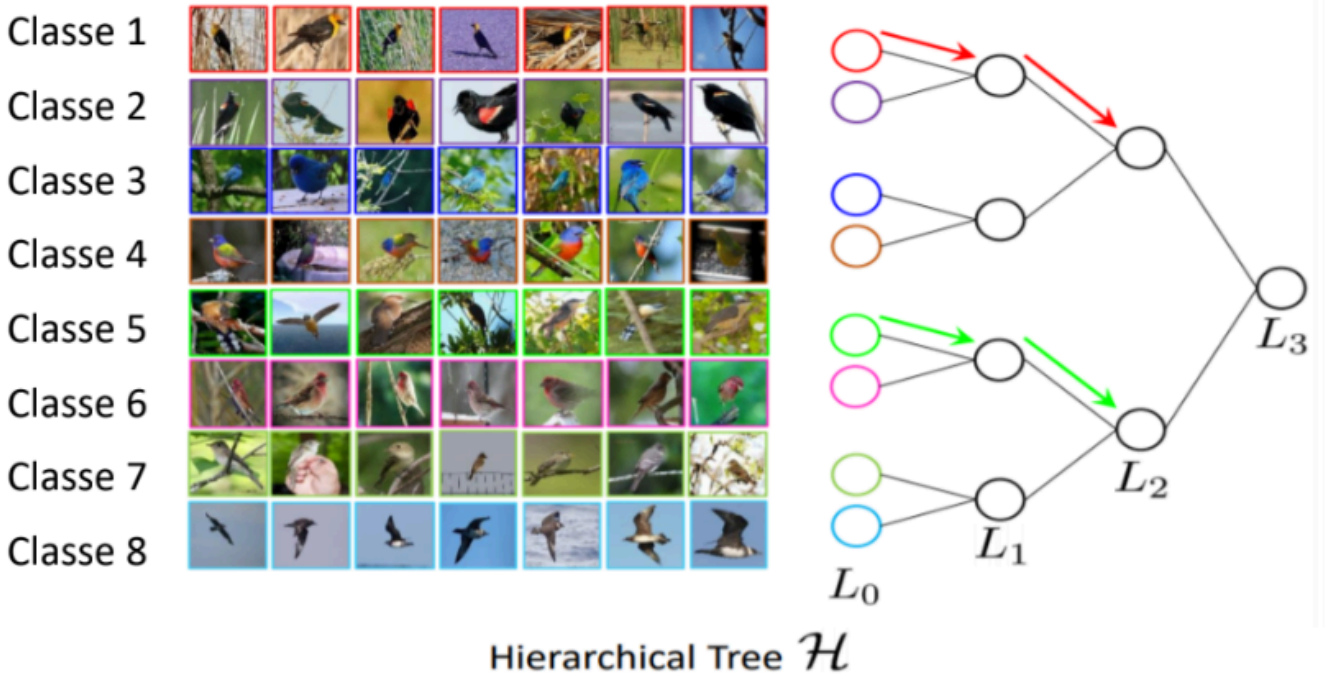


Figura 30: Exemplo de árvore de classes para seleção de triplas

Vamos definir o processo de escolha na árvore antes de estabelecermos como ela deve ser construída. O algoritmo consiste em 3 passos. O básico de conhecimento prévio que necessitamos é que, para cada **folha** da árvore, elas representam uma **classe**; e para cada **nó** da árvore, ele representa um **grupo de classes**.

```
function select_triplets(tree, batch_size) {
    seleciona l' classes aleatórias da árvore (folhas)
    for each âncora A {
        pego o positivo mais longe do âncora A (dentro da mesma classe)
        utilizo a árvore para achar as k classes vizinhas mais próximas da classe do âncora A e sorteio t negativos
    }
    return batch montado B ∈ ℝ^{l' × k × t}
}
```

Algoritmo 2: Exemplo de algoritmo para seleção de triplas na árvore

Agora que entendemos como e o porquê utilizamos a árvore, podemos compreender sua construção.

O primeiro passo para a construção da árvore, é a definição dos nó folha, onde cada um consiste em uma classe diferente do dataset. Depois, temos que definir a **between class distance**, que é a distância entre duas classes. Podemos utilizar, por exemplo:

$$d(p,q) = \frac{1}{n_p n_q} \sum_{i \in p} \sum_{j \in q} \|r_i - r_j\|^2 \tag{16}$$

onde r_k é o embedding da imagem k gerada pela rede neural.

Perfeito, agora precisamos definir uma medida de distância entre as imagens dentro da **mesma classe**, que chamaremos de **within class distance**. Podemos utilizar, por exemplo:

$$d_0 = \frac{1}{C} \sum_{c=1}^{C} \left(\frac{1}{n_c^2 - n_c} \sum_{i,j \in c} \|r_i - r_j\|^2 \right) \tag{17}$$

onde C é o número de classes e n_c é o número de imagens da classe c.

Agora, vamos fazer, recursivamente do nível $l = 0$ até $l = L$, uma **fusão** dos nós, de forma que os nós que satisfazerem um critério específico, vão ser unidos. Mas qual critério utilizamos?

$$d_l = \frac{l(4 - d_0)}{L} + d_0 \tag{18}$$

utilizamos dessa distância e, se duas classes possuem suas distâncias menores que d_l, elas são unidas. Esse processo se repete até o nível $l = L$.

2.5.2 Melhorar a Loss

Relembrando a nossa loss tripla, temos que:

$$\mathcal{L} = \sum_i \max(0, d(A_i, P_i) - d(A_i, N_i) + m) \tag{19}$$

O problema que temos nessa loss é que, por conta que a margem é fixa, muitas vezes, o modelo ignora triplas que podem ser de alguma forma úteis, seja por que a diferença da margem seja mínima, seja para exemplos mais difíceis mas que poderiam ajudar o modelo a compreender diferentes graus de parentesco entre as imagens ou variabilidade interna

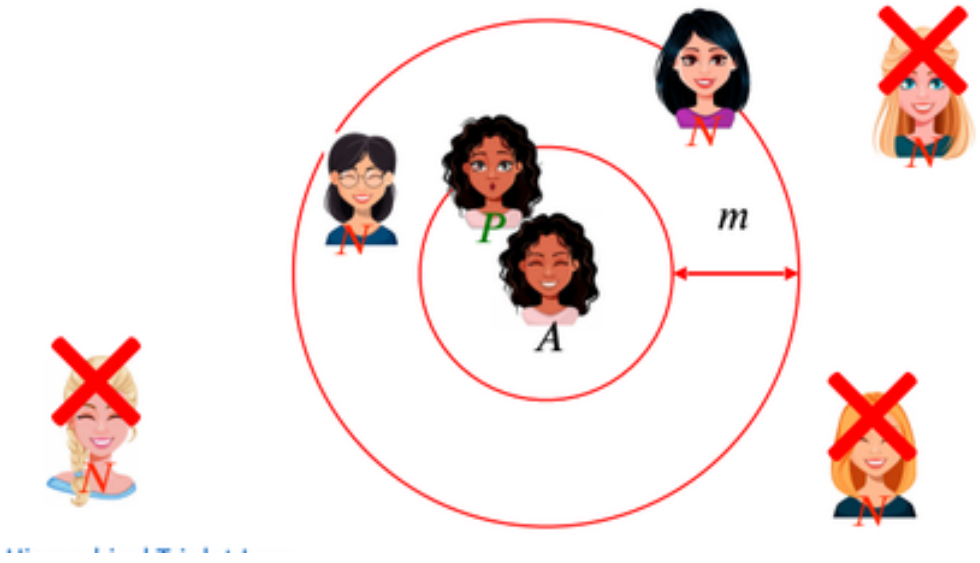


Figura 31: Exemplo da margem excluindo triplas úteis

A ideia é modificar a loss para que a margem fique dependente da distância entre a âncora e os exemplos negativos calculados na árvore hierárquica. Dessa forma, a loss final fica:

$$\mathcal{L} = \sum_i \max(0, d(A_i, P_i) - d(A_i, N_i) + m_i) \tag{20}$$

As distâncias mudam conforme o treino progride, logo, a árvore deve ser recalibrada a cada k épocas.

2.5.3 Ensemble Methods (Divide and Conquer)

Estamos trabalhando com um problema de redes neurais gerarem funções de um espaço $\mathbb{R}^n$ para um latente $\mathbb{R}^d$, de forma que, no espaço latente, os objetos parecidos fiquem próximos e os diferentes fiquem distantes. No entanto, a noção de parecido e diferente é **multifacetada**. Considere o caso dos rostos humanos, por exemplo, temos **cor de pele**, **formato do rosto**, **cor dos olhos**, **formato do nariz**, **formato da boca**, etc. Fazer com que **um único modelo** aprenda todas essas características pode sobrecarregar ele e atrapalhar seu aprendizado. A ideia do **dividir e conquistar** é que, em vez de uma única rede, combinamos múltiplos **learners** que juntam suas soluções em um espaço latente unificado.

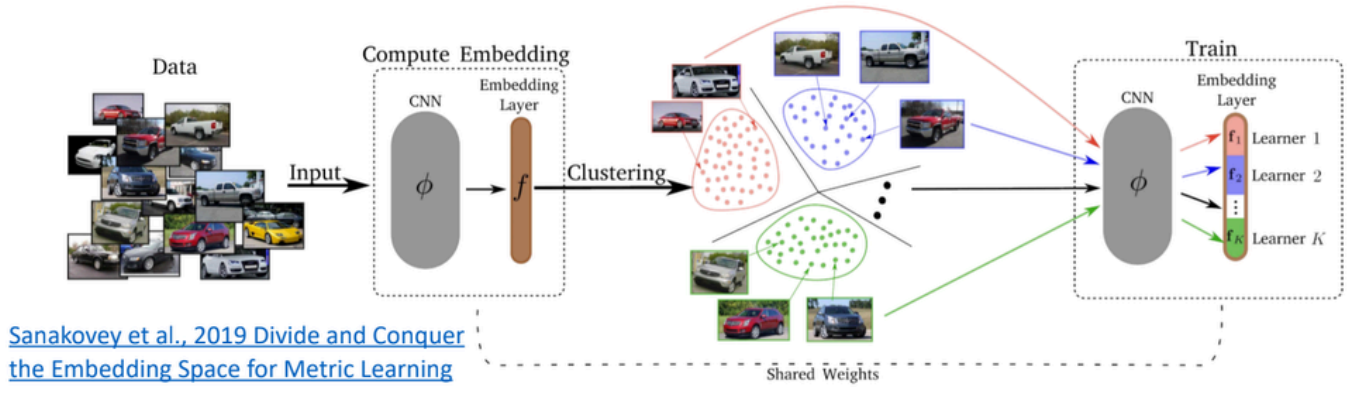


Figura 32: Exemplo de Multiple Learners

A primeira coisa que é feita é passar a imagem de entrada em uma CNN backbone **comum à todos os learners** (mesmos parâmetros em todos) que extrai as features brutas da imagem para que cada learner possa processar separadamente. Dos meus K learners, cada um tem sua própria camada linear que projeta a saída dessa backbone em um subespaço latente de dimensão d/K.

O algoritmo é dividido em duas fases, a **divisão** e a **conquista**. Na fase de divisão, temos 4 passos principais:

1. **Agrupamento (K-means)**: tilizando o espaço de embedding atual, as amostras do dataset são divididas em K clusters por meio do algoritmo K-Means. Cada cluster agrupa dados que compartilham noções de similaridade mais homogêneas (por exemplo, um cluster para picapes, outro para sedans, outro para SUVs)
2. **Definição dos K Learners**: Criam-se K learners, onde cada learner k é composto pela CNN compartilhada φ seguida pela sua projeção f_k para dimensão $\frac{d}{K}$
3. **Treinamento Alternado**: Os aprendizes são treinados alternadamente:
   - O aprendiz k é alimentado apenas com amostras do seu respectivo k-ésimo cluster.
   - Em cada retropropagação (backward pass), o gradiente atualiza os pesos da CNN compartilhada φ e os pesos da camada linear local f_k.
4. **Reagrupamento Periódico**: Como a representação de φ evolui com o treinamento, a cada T épocas todos os dados são re-clusterizados no espaço de embedding global atualizado

Já na fase de **conquista**, temos 2 passos principais:

1. **Fusão dos Aprendizes**: Após a convergência dos K aprendizes, as representações parciais (de dimensão $\frac{d}{K}$) são unidas/concatenadas para recompor o espaço de embedding completo de dimensão d
2. **Ajuste Fino Global (Fine-Tuning)**: Realiza-se um **fine-tuning** da camada de embedding em todo o conjunto de dados para garantir a **consistência mútua** entre as representações geradas individualmente por cada aprendiz

Aprendendo em Dados Não-Rotulados

3.1 Motivação

Infelizmente, o mundo real é um lugar cruel. O acesso a dados rotulados é muito limitado e, a não ser que você utilize de um trabalho manual humano massivo, ter um dataset com **muitas amostras** e **todas rotuladas** é praticamente impossível. O mundo real é cheio de dados não rotulados, então precisamos nos adaptar a isso.

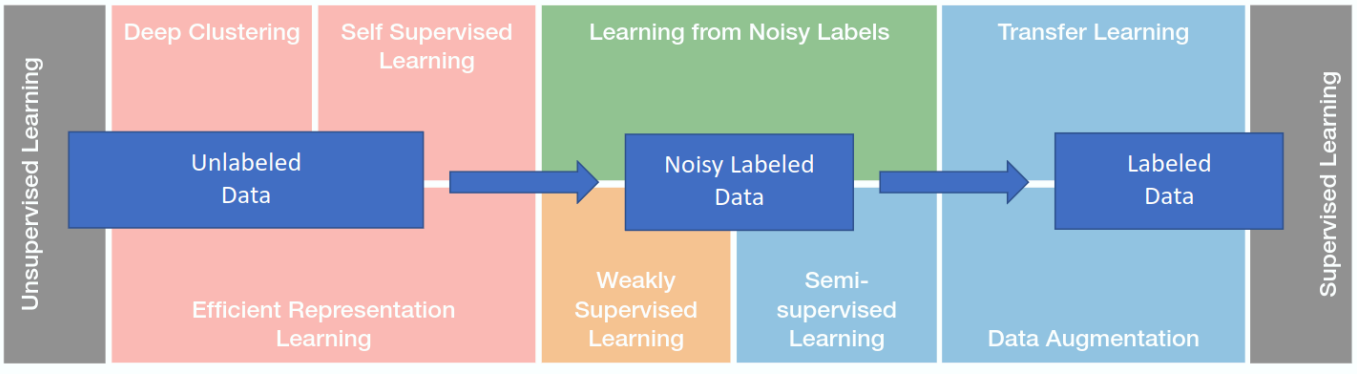


Figura 33: Espectro de Supervisão

3.2 Deep Clustering

A ideia do Deep Clustering é transformar os meus dados em uma representação mais simples (latente) e aplicar algoritmos de clustering para agrupar os dados. Para isso, utilizamos de uma arquitetura com **encoder** e **decoder**, de forma que fazemos um aprendizado inicial para que essa rede possa gerar uma representação latente dos dados. Depois, aplicamos um algoritmo de clustering (K-Means, por exemplo) para agrupar os dados.

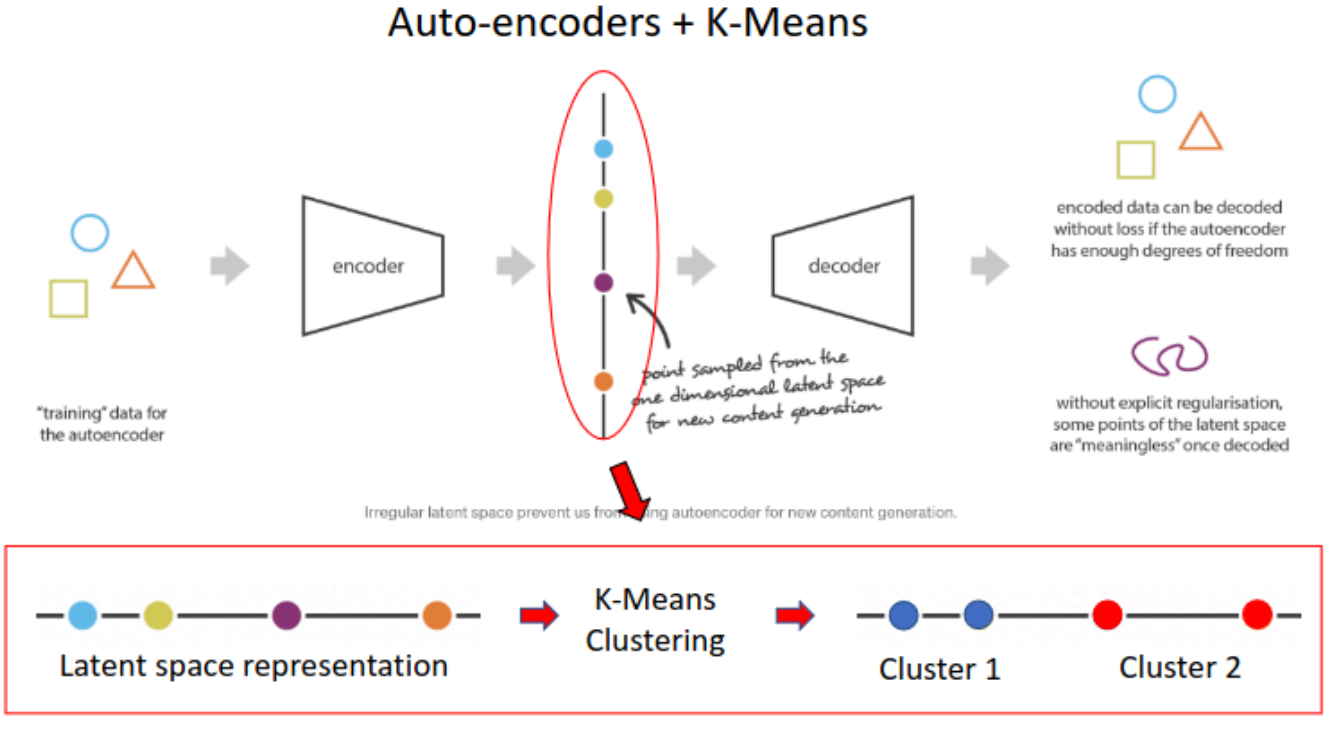


Figura 34: Arquitetura do Deep Clustering

3.2.1 Deep Embedding Clustering (DEC)

Essa abordagem é interessante, no entanto, existem algumas limitações. A principal delas, é o **hard assignment** do K-Means, que força os dados a serem agrupados em um cluster específico. Isso faz com que o modelo aprenda a gerar representações latentes que se encaixem nesse cluster, mesmo que ele não seja o mais adequado. O ideal é que a rede também aprenda a melhorar o

espaço latente de acordo com os agrupamentos, e para isso ambas as etapas devem ser feitas **ao mesmo tempo**, utilizando da retropropagação. Queremos isso pois não queremos que a rede coloque viezes no espaço latente. Por exemplo, o modelo pode aprender que, todas as imagens com um **deserto** no fundo são de **camelos**, mas isso **não é verdade**, pois existem outros animais que vivem em desertos. O ideal é que a rede aprenda a gerar representações latentes que separem os camelos de outros animais, mesmo que eles estejam no mesmo ambiente.

Só que o **hard assignment** do K-Means não é diferenciável, então não conseguimos fazer retropropagação. A solução para isso é utilizar de um **soft assignment**, onde cada ponto tem uma probabilidade de pertencer a cada cluster, e essa probabilidade é diferenciável. Seja z_i um ponto no espaço latente e μ_j o centro do cluster j, a probabilidade de z_i pertencer ao cluster j é dada por:

$$q_{ij} = \frac{\left(1 + \|z_i - \mu_j\|^2/\alpha\right)^{-\frac{\alpha+1}{2}}}{\sum_{j'} \left(1 + \|z_i - \mu_{j'}\|^2/\alpha\right)^{-\frac{\alpha+1}{2}}} \tag{21}$$

Perfeito! Pela lógica, se estamos estimando uma distribuição, precisamos de uma **distribuição alvo** para comparar com a distribuição estimada. No entanto, lembre-se que estamos mexendo em dados **não-rotulados**, então não temos uma distribuição alvo. E agora? O que fazemos?

A ideia é que **nós vamos criar nossa própria distribuição alvo**. A distribuição alvo é criada a **partir da distribuição estimada**, de forma que ela **reforce** os pontos que estão com alta probabilidade de pertencer a um cluster específico e **suaviza** os pontos que estão com baixa probabilidade. A distribuição alvo é definida como:

$$\begin{aligned} p_{ij} &= \frac{q_{ij}^2/f_j}{\sum_{j'} q_{ij'}^2/f_{j'}} \\ f_j &= \sum_i q_{ij} \end{aligned} \tag{22}$$

Elevar as probabilidades ao quadrado faz com que os pontos com alta probabilidade fiquem ainda mais altos, enquanto os pontos com baixa probabilidade fiquem ainda mais baixos. A divisão por f_j faz com que a distribuição alvo seja **balanceada**, ou seja, que cada cluster tenha aproximadamente o mesmo número de pontos e evite que clusters muito grandes dominem a distribuição alvo. Dessa forma, a rede aprende a gerar representações latentes que se encaixem melhor nos clusters, enquanto os clusters são ajustados de acordo com as representações latentes.

Agora, com o **pseudo-gabarito** criado, podemos definir a loss como a divergência de Kullback-Leibler entre a distribuição estimada e a distribuição alvo:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_i \sum_j p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) \tag{23}$$

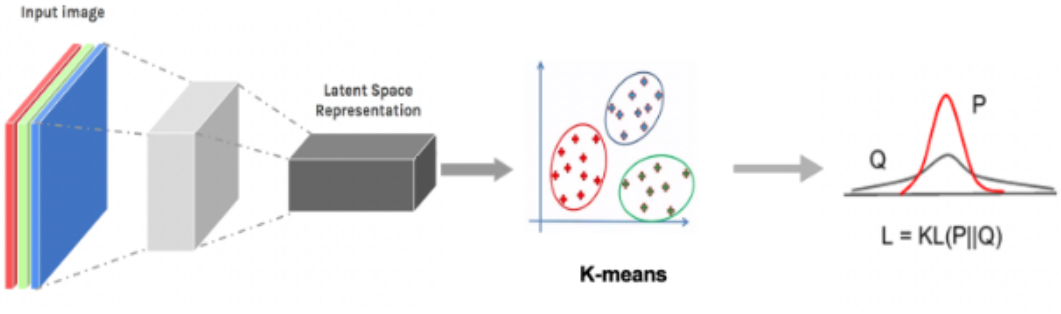


Figura 35: Arquitetura do Deep Embedding Clustering

O DCE pode então ser descrito em duas fases distintas:

1. **Pré-Treinamento**: Treina-se a rede para gerar representações latentes dos dados, utilizando de uma loss de reconstrução (por exemplo, MSE) para que o encoder possa gerar as representações úteis dos dados.
2. **Otimização**: Processo iterativo entre calcular a distribuição alvo P e a distribuição estimada Q, e atualizar os pesos da rede para minimizar a divergência entre as duas distribuições.

3.2.2 Deep K-Means

No K-Means tradicional, tentamos minimizar a distância entre os pontos z_i e seu centroide C_j. Seja μ_j o centroide do cluster C_j, no K-Means, minimizamos a seguinte função de erro:

$$\mathcal{L} = \sum_k \mathbb{I}\big[z_k \in C_j\big] \cdot \|z_k - \mu_j\|^2 \tag{24}$$

o problema aqui é a variável indicadora $\mathbb{I}\big[z_k \in C_j\big]$, que não é diferenciável. Para solucionar isso, o Deep K-Means substitui a variável indicadora por uma **escala de temperatura softmax** α, de forma que quanto mais próximo o ponto z_k estiver do centroide μ_j, maior será a probabilidade de pertencer ao cluster C_j. Essa função é dada por:

$$G_j(z_k;\alpha) = \frac{\exp\big(-\alpha\ \|z_k - \mu_j\|^2\big)}{\sum_{j'} \exp\big(-\alpha\ \|z_k - \mu_{j'}\|^2\big)} \tag{25}$$

Essa função é **diferenciável**. Definindo $f_\theta : \mathbb{R}^N \to \mathbb{R}^D$ como o encoder, $g_\theta : \mathbb{R}^D \to \mathbb{R}^N$ como o decoder, o Deep K-Means tenta minimizar a seguinte função de erro:

$$\begin{gathered} L = \sum_x L_{\text{rec}}(x) + \lambda L_{\text{clust}}(x) \\ L_{\text{rec}}(x) = \|x - g_\theta(f_\theta(x))\|^2 \qquad L_{\text{clust}}(x) = \sum_{k=1}^{K} G_k(f_\theta(x);\alpha)\ \|f_\theta(x) - \mu_k\|^2 \end{gathered} \tag{26}$$

Dependendo do valor de α, a função de distância dos clusterings pode se comportar de forma diferente

- $\alpha \to 0$: A função de distância se aproxima de uma distribuição uniforme, ou seja, todos os pontos têm a mesma probabilidade de pertencer a qualquer cluster.
- $\alpha \to 1$: A função de distância se aproxima de um soft assignment cru, ou seja, a probabilidade de pertencer a um cluster é proporcional à distância do ponto ao centroide.
- $\alpha \to \infty$: A função de distância se aproxima de um hard assignment, ou seja, cada ponto pertence a um único cluster.

O parâmetro é ajustado durante o treinamento (anneling), de forma que vai aumentando conforme o treino vai passando, fazendo com que as representações latentes posteriores se tornem mais certas de estarem próximas dos centróides corretos.

Deep K-Means Algorithm

function DeepKMeans(data $\mathcal{X}$, cluster's number K, balancing parameter λ, scheme for α epochs T, minibatches N, learning rate η) {
  for $\alpha = m_\alpha$ **to** M_α **do** {
    for $t = 1$ **to** T **do** {
      for $n = 1$ **to** N **do** {
        Draw a minibatch $\mathcal{X}_n \subset \mathcal{X}$
        Update $(\theta, \mathcal{R})$ using SGD
      }
    }
  }
}

3.3 Self-Supervised Learning

É uma forma de aprendizado onde os dados em si fornecem informações para a supervisão do aprendizado. Existe um panorama de aplicações e agrupamento pelos métodos que elas utilizam.

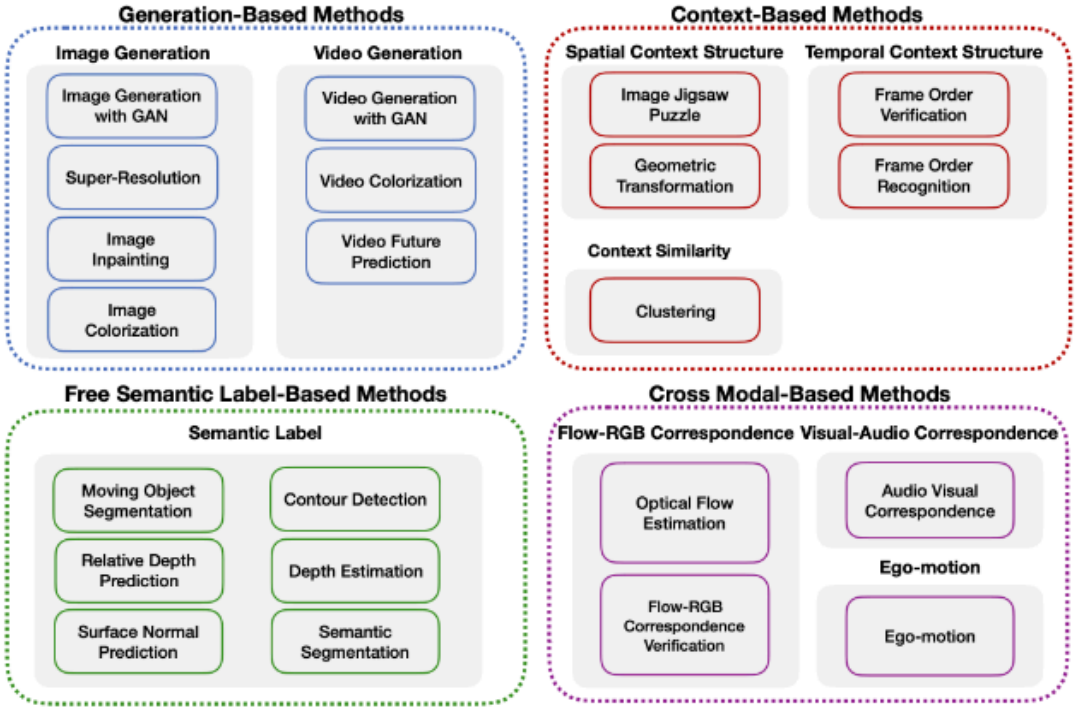


Figura 36: Espectro de métodos de Self-Supervised Learning

No entanto, essa abordagem encontra **muitos problemas** e **desafios** tanto práticos quanto teóricos dependendo da abordagem que está sendo implementada. Essas limitações são em grande parte superada nos próximos capítulos, mas vamos citar algumas delas:

- **Colapso de Representação (Representation / Dimensional Collapse)**: Sem rótulos externos para guiar o espaço latente, a rede pode encontrar "soluções triviais", mapeando todas as imagens para um vetor constante idêntico ou colapsando a representação em um subespaço de baixíssima dimensão.
- **Aprendizado de Atalhos (Shortcut Learning)**: O modelo pode aprender a resolver a tarefa pretexto explorando atalhos superficiais (como ruído de alta frequência, continuidade de bordas ou artefatos de textura) em vez de capturar conceitos semânticos abstratos reais.

- **Problema dos Falsos Negativos (False Negative Penalty)**: Em aprendizado contrastivo por pares (como o SimCLR), amostras distintas do mesmo batch são tratadas como pares negativos por padrão. Se duas imagens do batch pertencerem à mesma classe real (ex.: dois cachorros diferentes), o modelo é forçado incorretamente a afastar suas representações.
- **Custo Computacional e Batches Gigantescos**: Para garantir uma amostragem rica de negativos e estabilizar o treino, modelos contrastivos exigem mini-batches massivos (de 256 até 8192 amostras), demandando infraestruturas robustas de GPUs/TPUs e otimizadores específicos (como LARS).
- **Desconexão entre a Tarefa Pretexto e a Tarefa Final (Downstream Task Mismatch)**: As camadas finais adaptam-se fortemente ao objetivo pretexto (ex.: prever rotação ou pareamento), o que costuma exigir o descarte da camada de projeção (**projection head**) para aproveitar apenas as representações intermediárias.

3.4 Constrative Learning

A ideia do aprendizado contrastivo é utilizar de data augmentation para aproximar as representações de imagens que são **semânticamente** semelhantes e afastar as representações de imagens que são **semânticamente** diferentes. Para isso, pegamos uma imagem (original), uma versão dela aumentada (positiva) e uma versão de outra imagem (negativa). A rede neural recebe a imagem original e a versão aumentada e tenta gerar representações latentes próximas, enquanto que a representação da imagem negativa deve ser distante.

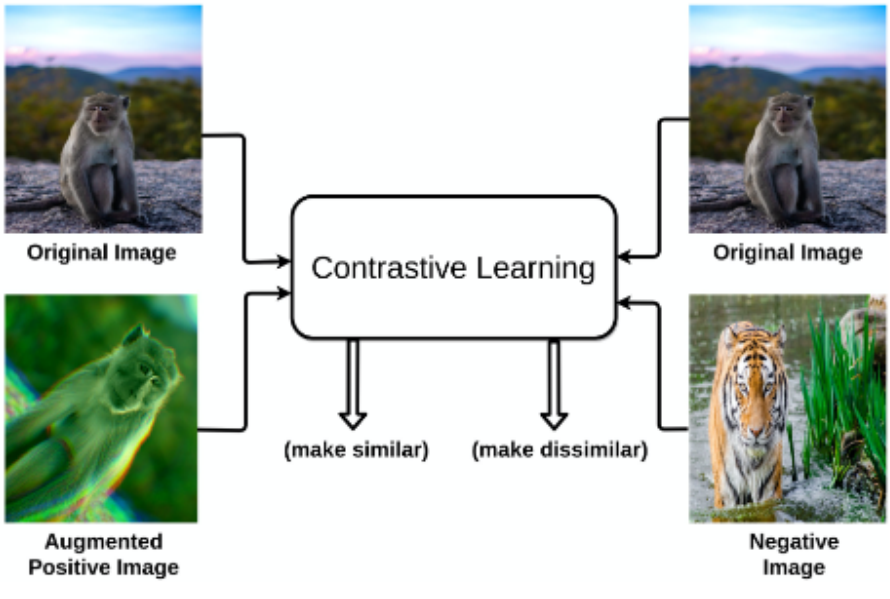


Figura 37: Ilustração do Aprendizado Contrastivo

A rede é então trainada para maximizar a concordância (agreement) entre as representações de imagens **positivas** e repelir (repel) as representações de imagens **negativas**.

3.4.1 Simple Framework for Contrastive Learning of Visual Representations (SimCLR)

Esse framework se baseia em 4 componentes sequenciais. Dada uma imagem x, aplicamos duas transformações (uma em cada) de uma família $\mathcal{T}$ e gero duas versões transformadas $\tilde{x}_i$ e $\tilde{x}_j$. Depois, passamos essas imagens em uma CNN $f(\cdot)$ para gerar as representações latentes $h_i = f(\tilde{x}_i)$ e $h_j = f\big(\tilde{x}_j\big)$. Em seguida, passamos essas representações em uma **projection head** $g(\cdot)$, uma pequena rede MLP com uma camada oculta e ativação não-linear ReLU que projeta h_i no espaço onde a **loss contrastiva** é calculada.

$$z_i = g(h_i) = W^{(2)}\sigma\big(W^{(1)}h_i\big) \tag{27}$$

É intuitivo pensarmos o porquê temos que adicionar essa **projection head**. Por que não fazer a loss já dentro do espaço latente h_i? A ideia é que, na nossa loss, vamos **forçar** com que as representações com transformações aplicadas sejam idênticas às originais, e para isso, utilizamos justamente a projection head para que a rede aprenda a perder informações irrelevantes para a tarefa de classificação. No entanto, posteriormente, podemos **descartar** a projection head e utilizar apenas o espaço latente h_i para as tarefas downstream.

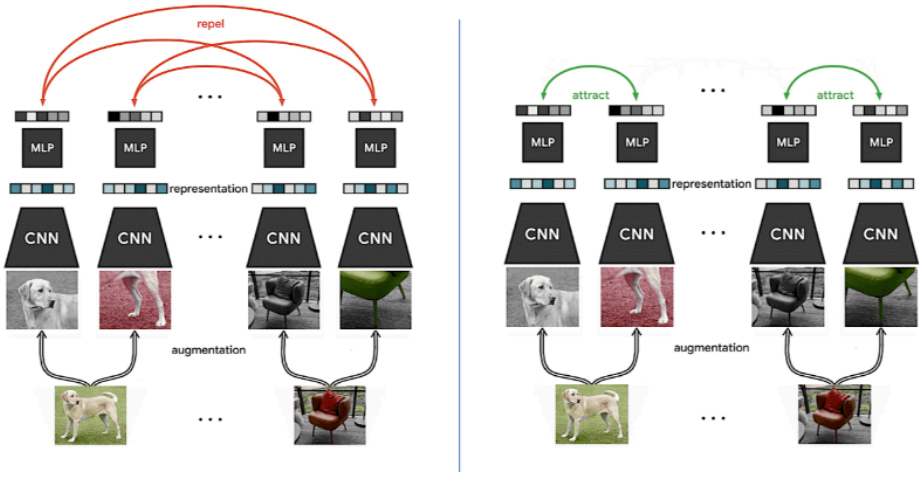


Figura 38: Arquitetura do SimCLR

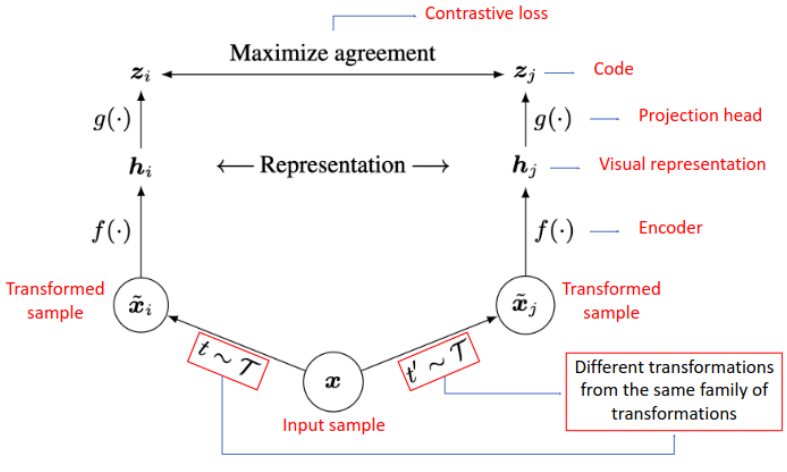


Figura 39: Ciclo do SimCLR

A transformação inicial $\mathcal{T}$ é composta por três augmentations simples sequenciais:

1. **Random Crop**: recorta aleatoriamente uma região da imagem e redimensiona para o tamanho original.
2. **Random Color Distortion**: aplica distorções de cor aleatórias, como ajuste de brilho, contraste, saturação e matiz.
3. **Random Gaussian Blur**: aplica um desfoque gaussiano aleatório à imagem, simulando variações de foco.

Nesse cenário, utilizamos a **Normalized Temperature-Scaled Cross Entropy Loss (NT-Xent)**, que é uma loss contrastiva baseada na entropia cruzada. Para um mini-batch de tamanho N, temos $2N$ amostras (cada imagem original gera duas amostras transformadas). As outras $2(N-1)$ amostras são tratadas como **negativas**. A loss para uma amostra positiva (i, j) é definida como:

$$\ell_{ij} = -\log\left(\frac{\exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)}{\sum_{k=1}^{2N} \mathbb{I}_{[k\neq i]} \exp\left(\frac{\text{sim}(z_i, z_k)}{\tau}\right)}\right) \tag{28}$$

onde sim representa a similaridade entre os vetores z_i e z_j, tipicamente calculada como a similaridade cosseno, e τ é um hiperparâmetro de temperatura que controla a suavidade da distribuição de probabilidade. A loss final é a média sobre todos os pares positivos no mini-batch:

$$L = \frac{1}{2N} \sum_{k=1}^{N} [\ell_{2k-1,2k} + \ell_{2k,2k-1}] \tag{29}$$

A lógica nessa loss, é que ao minimizar a loss, estamos maximizando a **similaridade** entre **representações positivas** e, por consequência, o **denominador** da loss, que representa a soma das similaridades entre a amostra positiva e todas as amostras negativas, é minimizado. Ou seja, estamos maximizando a **discrepância** entre **representações negativas**.

SimCLR Algorithm

function SimCLR(data $\mathcal{X}$, batch size N, constant τ, structure of f,g e $\mathcal{T}$) {
  for $\{x_k\}_{k=1}^{N} \subset \mathcal{X}$ {
    for all $k = 1$ **to** N {
      draw two augmentation functions $t, t' \sim \mathcal{T}$
      First Agumentation:
      $\tilde{x}_{2k-1} = t(x_k)$
      $h_{2k-1} = f(\tilde{x}_{2k-1})$
      Second Agumentation:
      $z_{2k-1} = g(h_{2k-1})$
      $\tilde{x}_{2k} = t'(x_k)$
      $h_{2k} = f(\tilde{x}_{2k})$
      $z_{2k} = g(h_{2k})$
    }
    for all $i = 1$ **and** $j = 1$ **to** $2N$ {
      $s_{ij} = z_i^T z_j / (\|z_i\| \, \|z_j\|)$
    }
    define $\ell(i,j)$ **as** $-\log\Big(\exp(s_{ij}/\tau) / \Big(\sum_{k=1}^{2N} \mathbb{I}_{[k \neq i]} \exp(s_{ik}/\tau)\Big)\Big)$
    define $\mathcal{L}$ **as** $\frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
    update f **and** g **to minimize** $\mathcal{L}$
  }
  throw away g
  return f
}

3.4.2 Aprendizado por Protótipos e Proporção de Rótulos (LLP)

Agora vamos pensar no cenário onde não temos os valores de rótulos das imagens, mas sabemos a proporção de cada classe dentro de um conjunto de imagens. Por exemplo, temos um conjunto de imagens com 1000 imagens, e sabemos que 30% são cachorros, 50% são gatos e 20% são pássaros. A ideia é que, mesmo sem saber quais imagens são de qual classe, podemos utilizar essa informação para guiar o aprendizado da rede neural.

A ideia inicial já vimos, utilizar de **contrastive learning** para aprender boas representações latentes das imagens. No entanto, o SimCLR necessita de batches enormes e não cria grupos, só afasta ou aproxima as representações. É aí que entra a nova abordagem do SwAV.

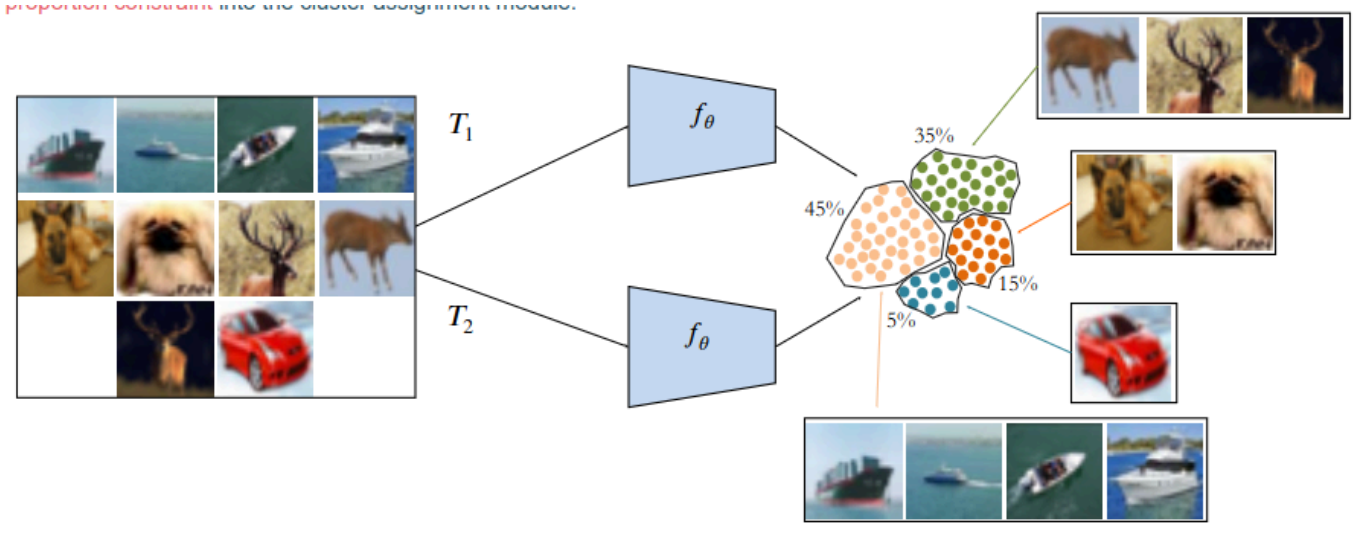


Figura 40: Ilustração do Aprendizado por Protótipos e Proporção de Rótulos (SwAV)

Em vez de apenas as features, o SwAV utiliza de **protótipos**. Utilizamos K protótipos $C = \{c_1, ..., c_K\}$. O modelo aprende que as duas versões da mesma imagem devem cair no mesmo protótipo.

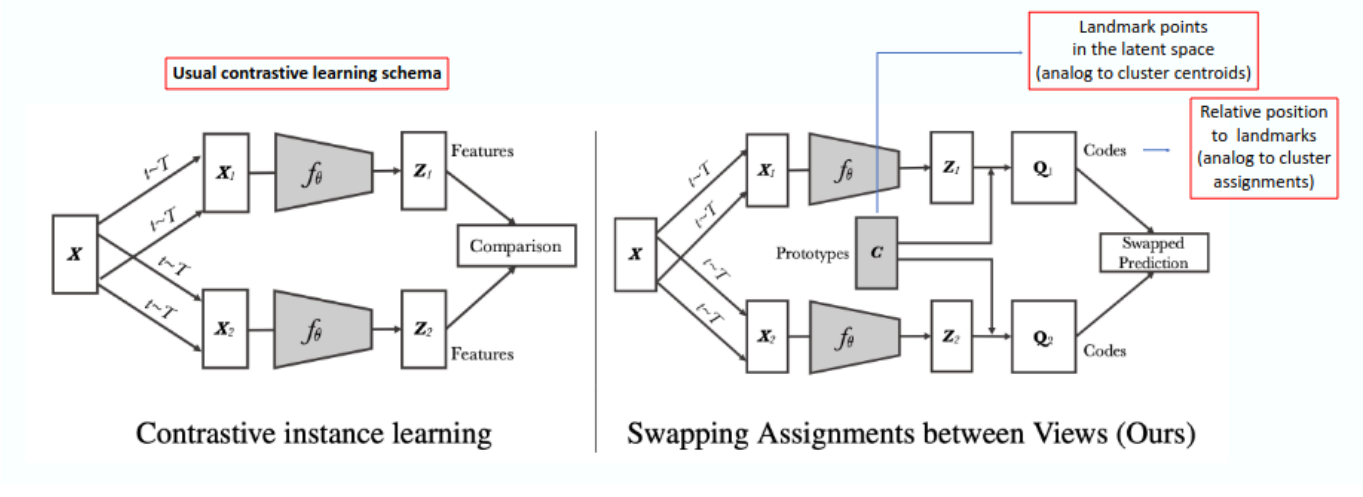


Figura 41: SimCLR v.s SwAV

O modelo SwAV consiste em um algoritmo de 5 passos:

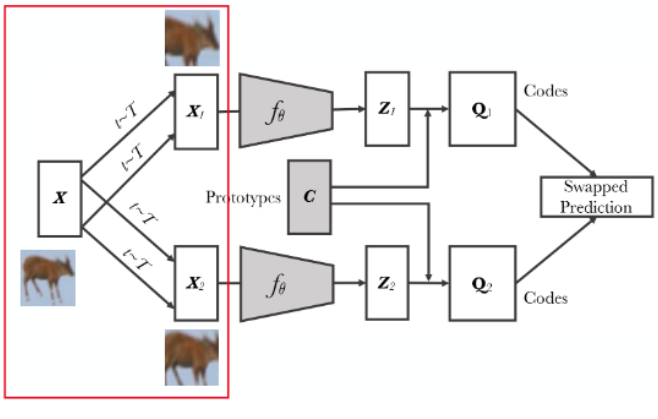


Figura 42: Highlight da parte de Data Augmentation do SwAV

- **Data Augmentation**: Para cada imagem x, aplicamos duas transformações aleatórias $\tilde{x}_1$ e $\tilde{x}_2$ da família de augmentations $\mathcal{T}$, gerando duas versões aumentadas da mesma imagem.

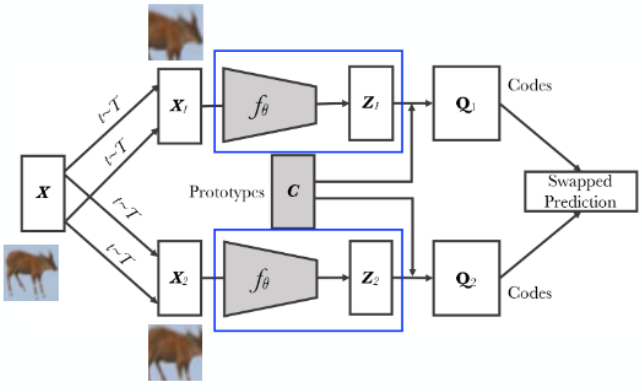


Figura 43: Hightlight da parte de Projeções do SwAV

- **Extração de Features**: Extraímos as features das imagens aumentadas utilizando uma CNN $f(\cdot)$, resultando em representações latentes $h_1 = f(\tilde{x}_1)$ e $h_2 = f(\tilde{x}_2)$. Como comportamente desejável, também tentamos limitar os valores das features pra ficarem dentro da esfera unitária.

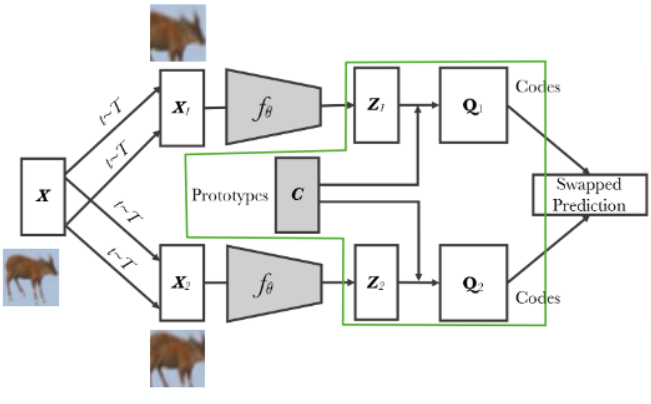


Figura 44: Hightlight da parte de Prototypes do SwAV

- **Formação de Protótipos**: Utilizamos as representações latentes para formar protótipos usando de **transporte ótimo**. Dadas B amostras $z_1, z_2, ..., z_B$, e os K protótipos $c_1, ..., c_K$, defina a matriz Z onde a j-ésima coluna é a representação latente z_j e a matriz C onde a k-ésima coluna é o protótipo c_k, então $C^T Z \in \mathbb{R}^{K \times B}$ guarda as **similaridades cosseno**. Com isso definido, precisamos de uma matriz $Q \in \mathbb{R}^{K \times B}$ que responda a seguinte pergunta: *Qual protótipo é o mais próximo de cada amostra?* Para isso, definimos a seguinte otimização:

$$Q^* = \text{argmax}_{Q \in \mathcal{Q}} \ \text{Tr}(Q^T C^T Z) + \varepsilon H(Q) \tag{30}$$

  o traço faz com que pesos maiores sejam assinalados para similaridades maiores, e a entropia faz com que a distribuição de pesos seja mais uniforme. Também limitamos os valores que Q pode assumir:

$$\mathcal{Q} = \left\{ Q \in \mathbb{R}^{K \times B} \mid Q 1_B = \frac{1}{K} 1_K, Q^T 1_K = \frac{1}{B} 1_B \right\} \tag{31}$$

  ela obriga as amostras a se dividirem igualmente entre os protótipos. Sem ela, a solução trivial seria todo mundo no mesmo cluster. ε controla o quanto a entropia vai suavizar a distribuição de pesos. A otimização é feita utilizando o algoritmo **Sinkhorn-Knopp** e da uma solução **aproximada fechada** para a matriz Q

$$Q^* = \text{diag}(u) \exp\left(C^T \frac{Z}{\varepsilon}\right) \text{diag}(v)$$
$$\text{diag}(u) = Q1_B \qquad \text{diag}(v) = Q^T 1_K \tag{32}$$

É nessa etapa que utilizamos das **proporções dos rótulos** para contribuir no aprendizado. Implementamos isso **modificando a restrição de equipartição**. Definimos o vetor w que tem as proporções de cada classe e modificamos a restrição de equipartição para:

$$\mathcal{C} = \left\{Q \in \mathbb{R}^{K \times B} \mid Q1_B = w, Q^T 1_K = \frac{1}{B}1_B\right\}$$
$$w = (w_1, w_2, ..., w_K)^T \qquad \sum_{k=1}^{K} w_k = 1 \tag{33}$$

a ideia é que os clusters gerados vão ser proporcionais aos rótulos que temos e podemos associar cada cluster a uma classe específica com base na proporção.

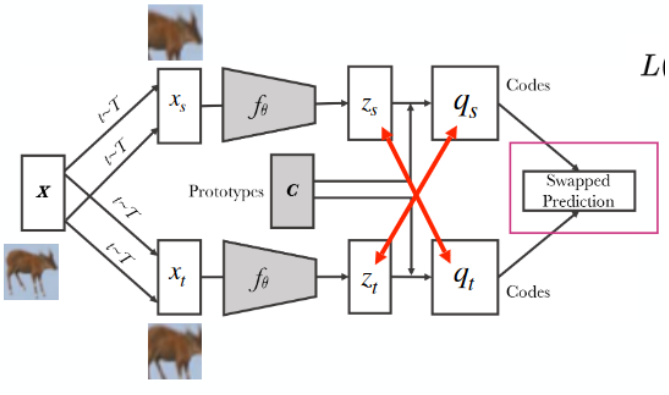


Figura 45: Hightlight da parte de Loss do SwAV

- **Swapped Prediction**: Cada view que geramos vai tentar prever o code da outra. A ideia é que, se z_t e z_s são duas views da mesma imagem, então a distribuição de protótipos q_t e q_s devem ser semelhantes. Definimos então a probabilidade de que a view z_t pertença ao protótipo c_k como:

$$p_{tk} = \mathbb{P}(z_t \in c_k) = \frac{\exp(z_t^T c_k / \tau)}{\sum_{k'} \exp(z_t^T c_{k'} / \tau)} \tag{34}$$

e a swapped loss como

$$\ell(z_t, q_s) = -\sum_{k=1}^{K} q_{sk} \log p_{tk}$$
$$L = \frac{1}{B} \sum_{b=1}^{B} [\ell(z_t, q_s) + \ell(z_s, q_t)] \tag{35}$$

- **Otimização**: O último passo é otimizar a loss final L

SwAV Algorithm

function SwAV(data $\mathcal{X}$, classes proportion w, epochs T, $\varepsilon > 0$) {
  for $t = 1$ **to** T {
    for each x_i **in** $\mathcal{X}$ {
      Generate two random views $X_i^{t,s}$
      Obtain the feature vectors $Z_i^{t,s}$
      Compute the prototype scores $C^T Z_i^{t,s}$
      Compute the codes $Q_i^{t,s}$ through Sinkhorn constrained to w
      Convert prototype scores to probabilities $P_i^{t,s}$
      Compute the swapped loss $L_i^{t,s} = -(Q_i^t \log(P_i^s) + Q_i^s \log(P_i^t))$
      Update θ and C with a gradient step to minimize $L_i^{t,s}$
      Normalize prototypes C
    }
  }
}

3.5 Aprendizado com Rótulos Ruidosos

Não bastando a dificuldade de anotar rótulos em dados reais, ainda existe o fator mais comum em tudo que fazemos nas nossas vidas, o **erro humano**. Em aplicações reais, pode acontecer comumente de que os rótulos anotados por humanos **esteja incorreto**.

3.5.1 SELFIE

No nosso dataset, cada amostra x tem um **rótulo verdadeiro** y (que não conhecemos) e um **rótulo anotado** $\tilde{y}$ (que é o que temos). O rótulo $\tilde{y}$ é **ruidoso** quando $y \neq \tilde{y}$. Definimos também a taxa de ruído η como a **probabilidade de um rótulo ser incorreto** $\tau = \mathbb{P}(\tilde{y} \neq y)$. Um modelo padrão possui o seguinte passo de atualização dos parâmetros em um minibatch:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \frac{1}{|B|} \sum_{x \in B}^{B} \mathcal{L}(f_\theta(x), \tilde{y}, \theta_t) \tag{36}$$

No entanto, dentro do SELFIE, fazemos a seguinte otimização:

$$\theta_{t+1} = \theta_t - \alpha \nabla \left(\frac{1}{|\mathcal{R} \cup \mathcal{C}|} \left(\sum_{x \in \mathcal{R}} \mathcal{L}\left(x, y^{\text{refurb}}; \theta_t\right) + \sum_{x \in \mathcal{C} \cap \mathcal{R}^{-1}} \mathcal{L}(x, \tilde{y}; \theta_t) \right) \right. \tag{37}$$

Aqui, a ideia é dividir o dataset em três grupos, as amostras confiáveis ($\mathcal{C}$), as que dá para consertar ($\mathcal{R}$) e as que não da pra confiar (são descartadas). Então utilizamos apenas as amostras confiáveis e as que foram consertadas e atribuídas com um novo rótulo y^{refurb}. No entanto, precisamos responder ainda três perguntas:

Como escolher $\mathcal{C}$?: Utilizamos do **Lowest loss-based separation method**, onde selecionamos as $\tau \cdot 100\%$ amostras com menor loss. A ideia é que, quando a loss é alta, significa que o modelo não está entendendo que aquele dado pertence naquela classe, então provavelmente o rótulo está incorreto. O τ é empírico ou estimado por validação cruzada.

SELFIE Algorithm

function SELFIE(data $\mathcal{D}$, epochs T, γ warm-up, noise rate τ, uncertainity threshold ε, history length q, how many times the model will restart r) {
  $\mathcal{R}^{\text{tot}} \leftarrow \varnothing$
  for $r = 1$ **to** r {
    $p \leftarrow 1$
    Initialize θ_p
    for $t = 1$ **to** T {
      for $j = 1$ **to** $|\mathcal{D}|/|\mathcal{M}|$ {
        Draw minibatch $M \subset D$
        Warm-up periods:
        if $i \leq \gamma$ {
          $\theta_{p+1} = \theta_p - \alpha \nabla \left(\frac{1}{|\mathcal{M}|} \sum_{x \in \mathcal{M}} \mathcal{L}\left(x, \tilde{y}; \theta_p\right) \right)$
        } **else** {
          $\mathcal{C} \leftarrow (1 - \tau) \cdot 100\%$ of low loss samples in $\mathcal{M}$
          $\mathcal{R} \leftarrow \varnothing$
          for each x **in** $\mathcal{M}$ {
            if $F(x, q) \leq \varepsilon$ **or** $x \in \mathcal{R}^{\text{tot}}$ {
              $\mathcal{R} \leftarrow \mathcal{R} \cup \left(x, y^{\text{refurb}}\right)$
            }
          }
          $\mathcal{R}^{\text{tot}} \leftarrow \mathcal{R}^{\text{tot}} \cup \mathcal{R}$
          $\theta_{p+1} = \theta_p - \alpha \nabla \left(\frac{1}{|\mathcal{R} \cup \mathcal{C}|} \left(\sum_{x \in \mathcal{R}} \mathcal{L}\left(x, y^{\text{refurb}}; \theta_p\right) + \sum_{x \in \mathcal{C} \cap \mathcal{R}^{-1}} \mathcal{L}\left(x, \tilde{y}; \theta_p\right) \right) \right)$
        }
      }
    }
    $p \leftarrow p + 1$
  }
  return θ_p, $\mathcal{R}^{\text{tot}}$
}

Como escolher $\mathcal{R}$?: Para cada amostra do modelo, guardamos o histórico $H_x(q)$ das últimas q previsões do modelo. Definimos então a **probabilidade** de que o rótulo correto seja j dado que minha amostra é x e que eu fiz q predições como:

$$\mathbb{P}(j \mid x, q) = \frac{\sum_{\tilde{y} \in H_x(q)} \mathbb{I}[\tilde{y} = j]}{|H_x(q)|} \tag{38}$$

ou seja, a proporção de vezes que o modelo previu a classe j para a amostra x. Definimos então a **incerteza** dessa previsão como a **entropia normalizada** dessa distribuição que acabamos de definiir:

$$F(x;q) = \left(\frac{1}{\delta}\right) \operatorname{entropy}(\mathbb{P}(y \mid x, q))$$

$$\operatorname{entropy}(\mathbb{P}(y \mid x, q)) = -\sum_{j=1}^{C} \mathbb{P}(j \mid x, q) \log \mathbb{P}(j \mid x, q) \tag{39}$$

$$\delta = -\sum_{j=1}^{C} \left(\frac{1}{C}\right) \log\left(\frac{1}{C}\right)$$

onde o δ serve para normalizar e a incerteza ficar entre 0 e 1. Com essas definições, finalmente podemos criar a definição de **o que é uma amostra consertável**. Definimos uma amostra como **refurbishable** quando a incerteza preditiva (F) é tal que:

$$F(x;q) < \varepsilon \quad (0 < \varepsilon < 1) \tag{40}$$

Como corrigir as amostras consertáveis?: Para as amostras com rótulos erados, vamos assinalar os rótulos **que o modelo mais previu dentro do histórico de q predições**.

$$y^{\text{refurb}} = \operatorname{argmax}_{1 \le j \le C} \mathbb{P}(j \mid x, q) \tag{41}$$

3.6 Aprendizado Fracamente Supervisionado

Vamos supor que temos um conjunto de dados com N amostras. Vamos supor também que nos foi dada a tarefa de aplicar um modelo de **segmentação semântica** no meu conjunto. Parece uma tarefa tranquila! Mas quando você vai olhar os dados, eles não tem máscaras que indicam a posição real dos objetos, mas sim, pontos onde os objetos ficam... E agora? Será que existe uma forma de ainda fazer um modelo que aprende o formato dos objetos mesmo que meu ground truth seja só o ponto do centro do objeto e mais nada? É aí que entra o aprendizado fraco supervisionado.

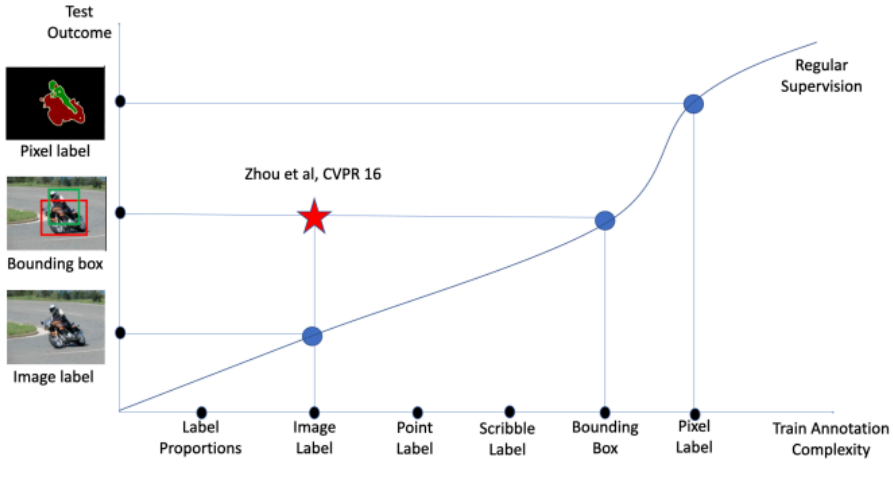


Figura 46: Gráfico comparativo da saída do teste em função da complexidade do rótulo fornecido

O ponto do aprendizado fracamente supervisionado é você utilizar rótulos **menos informativos** do que a tarefa que você vai resolver exige.

3.6.1 Class Activation Mapping (CAM)

Como exemplo de modelo que utiliza dessa abordagem, o CAM se destaca bastante. Ele utiliza apenas de labels anotadas e como saída gera um **bouding box** que indica a posição do objeto na imagem.

A técnica utilizada é treinar uma rede de classificação e, dentro dos seus feature maps, localizar **onde** ela olhou para gerar o rótulo de classificação.

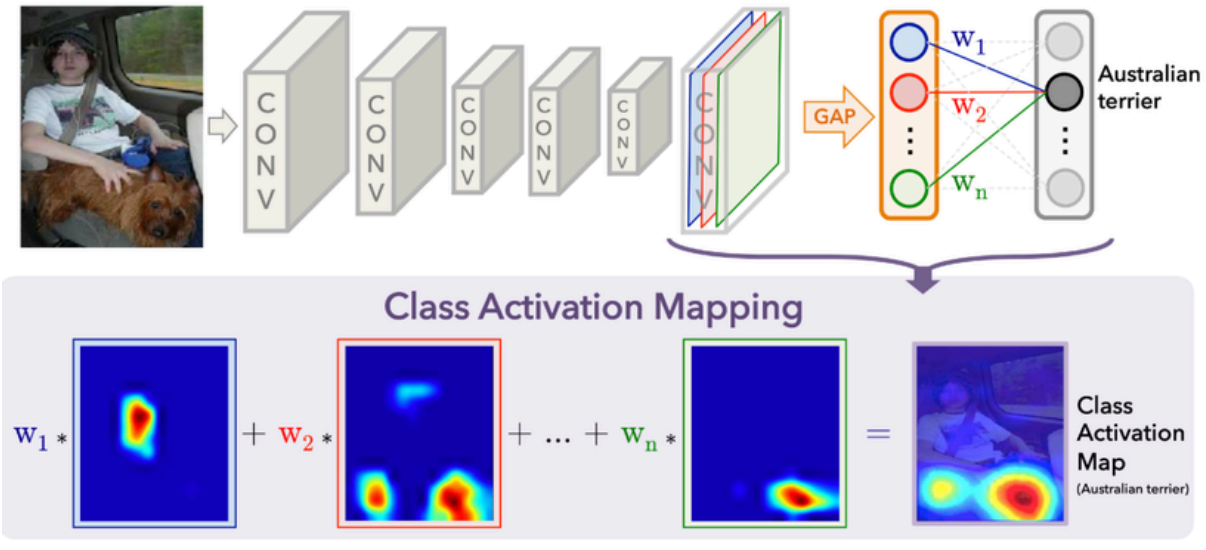


Figura 47: Arquitetura do CAM

Vamos definir w_k^c como o peso que liga o mapa k à classe c na camada de classificação. Ou seja, ele dita o quão importante aquela **feature** específica influencia na decisão de classificar a imagem como pertencente à classe c. Definimos também $f_k(i,j)$ como o valor do feature map k na posição (i,j) **antes** do **global average pooling** (GAP). Defina também $F_k = \sum_{i,j} f_k(i,j)$, de forma que o score da classe C na sa´ida softmax é dada por

$$S_c = \sum_k w_k^c F_k = \sum_k w_k^c \sum_{i,j} f_k(i,j) = \sum_{i,j} \sum_k w_k^c f_k(i,j) \tag{42}$$

Se pegarmos a parte interna da soma e chamarmos de $M_c(i,j) = \sum_k w_k^c f_k(i,j)$, então $M_c(i,j)$ representa a importância da feature na posição (i,j) para a classe c, então posições com M_c alto puxam a probabilidade da classe c para cima.

Para selecionar a bouding box, pegamos a região $M_c(i,j)$ maior que 20% do valor máximo de $M_c(i,j)$ e escolhemos a componente conexa maior. Essa região é a nossa bouding box. Fazemos isso para as 5 classes mais prováveis.

3.7 Aprendizado Semi-Supervisionado

Imagine agora que nós temos um conjunto com várias imagens e precisamos fazer uma detecção de objetos (aplicar bouding boxes e ver onde estão os objetos). Começamos com os dados sem rótulo nenhum, mas isso não é problema! Você tirou algum tempo e criou as bouding boxes reais de algumas imagens, mas são muitas... Será que da pra aproveitar o pouco que você fez e ainda sim conseguir prever do resto que você ainda não tem rótulo? É aí que entra o aprendizado semi-supervisionado.

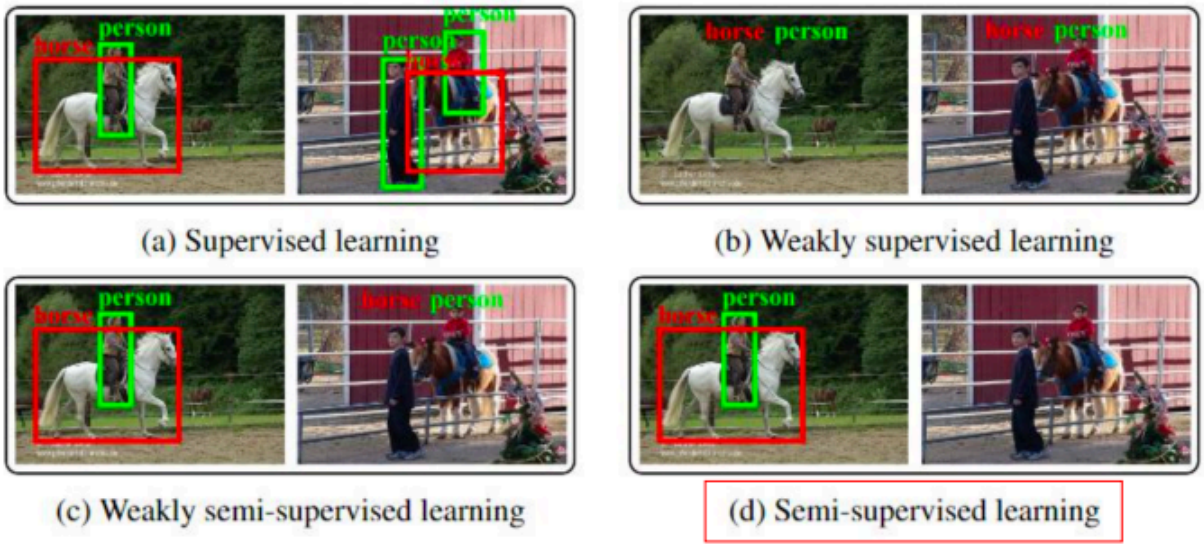


Figura 48: Tipos de aprendizado

3.7.1 FixMatch

O artigo do FixMatch diz que ele é simplesmente a unificação de dois conceitos já bastante aplicados:

1. **Regularização por consistência:** o modelo deve dar predições parecidas para versões perturbadas da mesma imagem;
2. **Pseudo-rotulagem:** o próprio modelo gera rótulos artificiais para os dados sem rótulo, e depois treina com eles.

Ela gera duas versões, uma com augmentation fraca (por exemplo flip e pequenos deslocamentos) e outra com augmentation forte (distorções agressivas de cor e forma). A versão fraca passa pelo modelo e produz uma predição. Se essa predição for confiante, vira um pseudo-rótulo. A versão forte passa pelo mesmo modelo, e a cross-entropy $H(p, q)$ força a predição da versão forte a coincidir com o pseudo-rótulo.

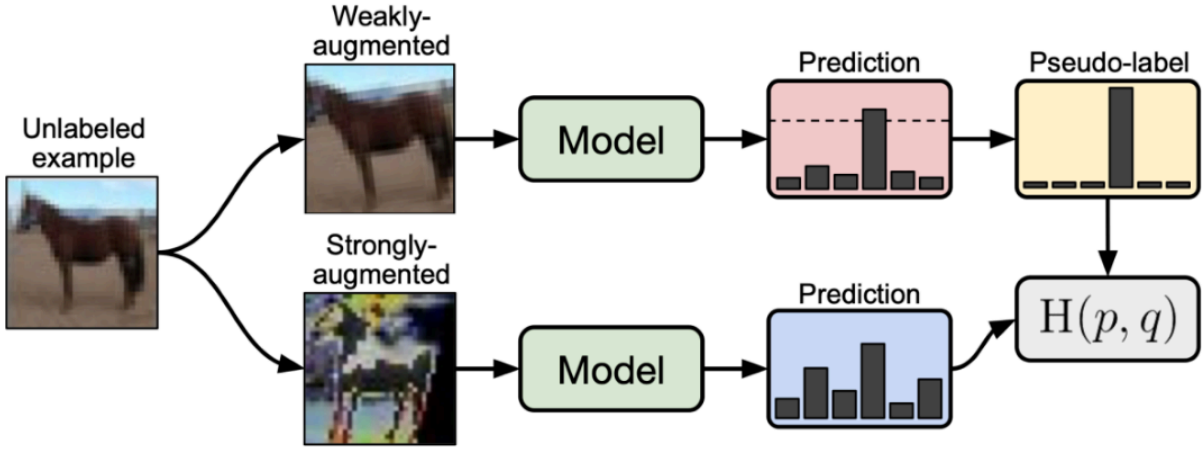


Figura 49: Arquitetura do FixMatch

A intuição é que a versão fraca é mais fácil, então a predição dela é mais confiável. Ela ensina a versão forte, que é difícil de classificar. Assim o modelo aprende a ser invariante a perturbações pesadas.

A loss geral é a soma de duas

$$L = \ell_s + \lambda \ell_u \tag{43}$$

ℓ_s é a loss sobre os meus dados rotulados com augmentation fraca, e ℓ_u é a loss sobre os meus dados não rotulados com augmentation forte. Em questão de notação, considere α como a função de augmentation fraca e $\mathcal{A}$ como a função de augmentation forte.

$$\begin{aligned} \ell_s &= \frac{1}{B}\sum_{b=1}^{B} H(p_b, p_m(y|\alpha(x_b);\theta)) \\ \ell_u &= \frac{1}{\mu B}\sum_{b=1}^{\mu B} \mathbb{I}[\max(p_m(y|\alpha(u_b);\theta))] \geq \tau] H(\hat{p_b}, p_m(y|\mathcal{A}(u_b);\theta)) \end{aligned} \tag{44}$$

FixMatch Algorithm

function FixMatch(labeled batch $\mathcal{X} = \{(x_b, p_b) : b = 1, ..., B\}$, unlabeled batch $\mathcal{U} = \{(u_b) : b = 1, ..., \mu B\}$, confidence threshold η, unlabeled data ratio μ, unlabeled loss weight λ_b) {
  $\ell_s = \frac{1}{B}\sum_{b=1}^{B} H(p_b, p_m(y|\alpha(x_b);\theta))$
  for $b = 1$ **to** μB {
    Strong augmentation in u_b:
    $\tilde{u}_b = \mathcal{A}(u_b)$
    $q_b = p_m(y|\alpha(u_b);\theta)$
  }
  $\ell_u = \frac{1}{\mu B}\sum_{b=1}^{\mu B} \mathbb{I}[\max(q_b) \geq \tau] H(\text{argmax}(q_b), \tilde{u}_b)$
  return $\ell_s + \lambda_b \ell_u$
}

Self-Attention

4.1 Introdução

Um conceito amplamente estudado pela psicologia é a capacidade que o ser humano tem de **prestar atenção apenas naquilo que lhe interessa**. Um belo exemplo já documentado é o **Efeito Coquitel** (cocktail effect), onde em uma festa com muito barulho, conseguimos prestar atenção em uma conversa específica e ignorar o resto.

Com base nesse conceito de atenção, será que conseguimos fazer com que redes neurais consigam prestar atenção apenas em informações relevantes para uma determinada tarefa? A resposta é sim, e é aí que entra o conceito de **Self-Attention**. Resumidamente, podemos definir a atenção como **um conjunto de pesos que diz o quanto cada parte da entrada importa**.

No entanto, a atenção vai muito além, por conta do que chamamos de **contexto**. Dependendo de **o que está no input** e **como está organizado**, os pesos associados para cada parte da entrada pode mudar. Vamos tomar como exemplo a frase *"I swam across the river to get to the other side of the bank"*. A nossa tarefa é classificar qual é o significado da palavra *bank* na frase. A palavra *bank* pode significar *banco* ou *margem do rio*, e para descobrir qual é o significado correto, precisamos olhar para o contexto da frase.

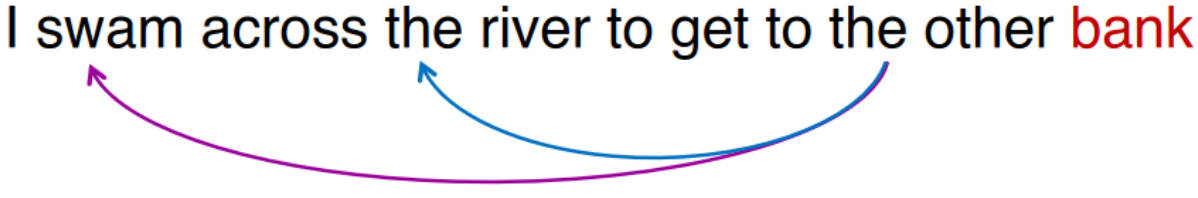


Figura 50: Ilustração do Self-Attention

Nesse caso, podemos ver como as palavras *swam* e *river* são mais importantes para determinar o significado da palavra *bank*, enquanto que as palavras *get* e *other* são menos importantes. Ou seja, a atenção é **contextual**, e depende de **o que está no input** e **como está organizado**.

Outro bom exemplo é a utilização de mecanismos de atenção em **séries temporais**. Dada uma série temporal ruidosa $x = [x_1, x_2, ..., x_n]$, $w = [w_j]$ é um vetor de pesos e queremos utilizar do mecanismo de atenção para **remover a influência dos ruídos**. Para isso, podemos modificar nossos pesos de forma que, quanto mais distante esteja o valor da série do meu i atual, menor sua influência

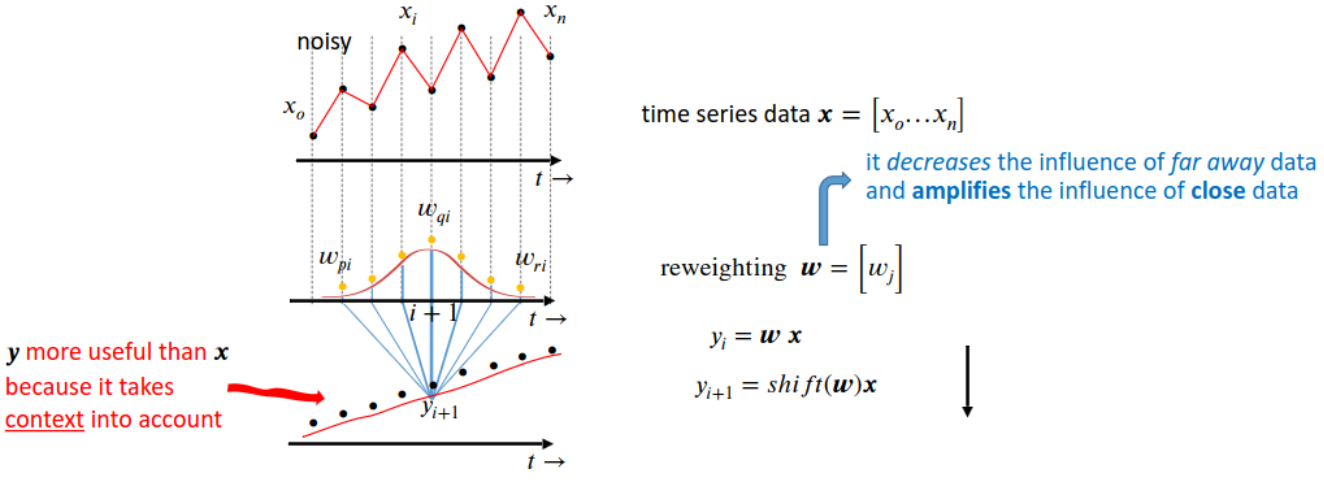


Figura 51: Ilustração do Self-Attention em séries temporais

4.2 Self Attention para Processamento de Linguagem Natural

A ideia aqui é de transformar nossas palavras em uma representação vetorial, que permite realizar operações e comparações. Por um exemplo, nessa forma vetorial, poderiamos fazer algo do tipo:

$$\text{King} - \text{Man} + \text{Woman} = \text{Queen} \tag{45}$$

Melhor ainda, utilizando dessas representações vetoriais, podemos aplicar operações para contribuir com **mais contexto** para minha palavra. Vamos tomar por exemplo as seguintes frases:

- *A manga da minha camisa sujou*
- *Tirei uma manga do pé ontem e comi na salada de frutas*

Aqui, se transformássemos manga em uma representação vetorial, ela seria a mesma para as duas frases. No entanto, o contexto da frase é diferente, e a palavra *manga* tem significados diferentes. Então utilizando de pesos e operações entre as representações vetoriais de cada uma das palavras, seria possível obter uma representação vetorial diferente para cada uma das palavras *manga* dependendo do contexto da frase. Nesse cenário, por exemplo, o peso que liga *manga* com *camisa* sera **muito forte**, pois a palavra manga contribui para indicar o significado da palavra *manga* na frase.

Dentro dos nossos mecanismos, precisamos de uma ferramenta para medir a similaridade entre o embedding de duas palavras. Para isso, vamos utilizar do produto escalar (dot product) entre os embeddings das palavras. A intuição aqui é que o **tamanho** do vetor indica a força semântica dele, enquanto a direção indica o significado. Se dois vetores são grandes e estão próximos, o seu produto interno vai ser grande, e o contrário se eles forem muito diferentes.

4.2.1 Mecanismo Inicial

Vamos imaginar um mecanismo bem básico dado tudo que falamos. Vamos pensar numa frase com 4 palavras (podem ser n). Eu quero saber o peso de importância para o significado da **palavra 3** na frase (que foi transformada no vetor v_3), ela é a chamada **query**. Eu faço então o produto interno com cada uma das outras palavras da frase (que foram transformadas nos vetores v_1, v_2, v_3 e v_4), que são as chamadas **keys**. O resultado do produto interno é um vetor de pesos não normalizados $s = [s_1, ..., s_4]$. Eu então normalizo esses pesos (com a normalização softmax) para que fiquem entre 0 e 1 e somem 1.

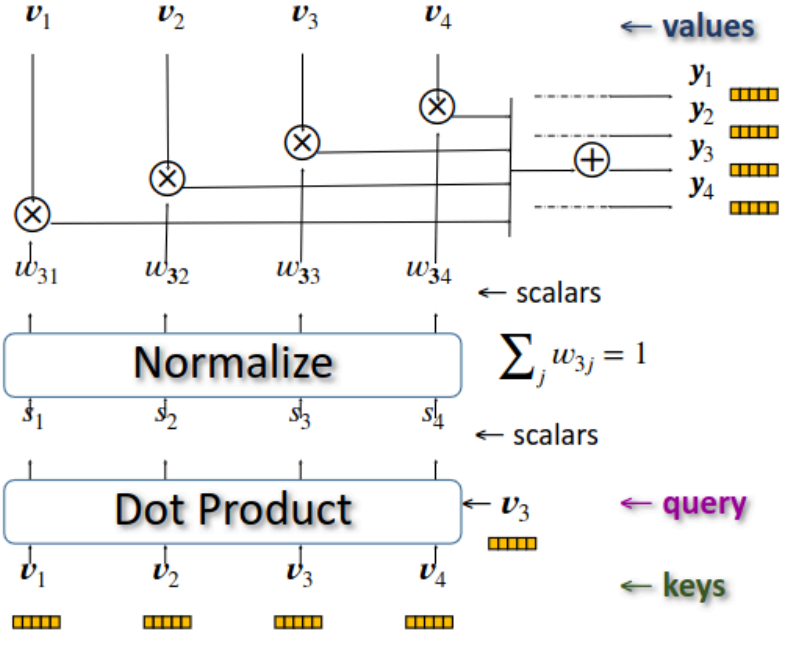


Figura 52: Ilustração do mecanismo inicial de Self-Attention

Então fazemos uma multiplicação entre pesos normalizados e os vetores das palavras (que são as chamadas **values**), e somamos o resultado. O resultado final é um vetor que representa a palavra 3 com mais contexto, pois ele leva em consideração o significado das outras palavras da frase.

Resumidamente e de forma mais matemática e genérica, podemos resumir dessa forma:

1. Conjunto de embeddings de palavras: $V = \{v_1, ..., v_n\}$
2. Palavra de interesse: v_i
3. Fazemos o produto interno de v_i com cada um dos embeddings (inclusive o próprio embedding da query) de palavras:

$$S_i = \left(s_{i1}, ..., s_{in} : s_{ij} = v_i^T v_j\right) \tag{46}$$

4. Pegamos os pesos gerados e normalizamos:

$$W_i = \left(w_{i1}, ..., w_{in} : w_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^{n} \exp(s_{ik})}\right) \tag{47}$$

5. Geramos os novos embeddings contextualizados a partir dos pesos

$$y_i = \sum_{j=1}^{n} w_{ij} v_j \tag{48}$$

Algo interessante de fazermos nesse momento é formalizar isso tudo em notações matriciais! Isso vai ajudar a gente a entender todos os conceitos mais complexos e notações matemáticas posteriores.

Se nosso conjunto de embeddings de palavras é $V = \{v_1, ..., v_n\}$, podemos organizar esses vetores em uma matriz $V \in \mathbb{R}^{n \times d}$, onde cada linha é um embedding de palavra. Com essa notação, podemos obter uma matriz S tal que:

$$S = VV^T = \begin{pmatrix} - & v_1^T & - \\ & \vdots & - \\ - & v_n^T & - \end{pmatrix} \begin{pmatrix} | & & | \\ v_1 & \dots & v_n \\ | & & | \end{pmatrix} = \begin{pmatrix} v_1^T v_1 & \dots & v_1^T v_n \\ \vdots & \ddots & \vdots \\ v_n^T v_1 & \dots & v_n^T v_n \end{pmatrix} = \begin{pmatrix} s_{11} & \dots & s_{1n} \\ \vdots & \ddots & \vdots \\ s_{n1} & \dots & s_{nn} \end{pmatrix} \tag{49}$$

Dessa forma, a matriz S é uma matriz de similaridade entre os embeddings das palavras. Cada elemento s_{ij} representa a similaridade entre a palavra i e a palavra j e a linha k da matriz S representa a similaridade da palavra k com todas as palavras da frase (com ela mesma inclusa). Depois aplicamos $\mathrm{softmax}$ na matriz S para obter a matriz de pesos W (linha a linha).

$$W = \begin{pmatrix} w_{11} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{n1} & \dots & w_{nn} \end{pmatrix} \tag{50}$$

Por fim, obtemos as novas representações enriquecidas com contexto multiplicando a matriz de pesos W pela matriz de embeddings V:

$$Y = WV \tag{51}$$

onde a linha i de Y é $y_i^T = \sum_{j=1}^{n} w_{ij} v_j^T$

Perceba, no entanto, o problema desse mecanismo inicial. Ele não possui **pesos aprendíveis**, ou seja, não permite que minha máquina aprenda!

4.2.2 Mecanismo Aprendível

Perceba que nesse nosso mecanismo, tanto os **values**, quanto os **keys** e as **queries** são os mesmos embeddings das palavras. Mas seria interessante que a máquina conseguisse modificar

todos esses valores de acordo com o que ela aprendeu. Para isso, vamos criar três matrizes de pesos aprendíveis $M_Q \in \mathbb{R}^{d\times d}$, $M_K \in \mathbb{R}^{d\times d}$ e $M_V \in \mathbb{R}^{d\times d}$, além de renomear a nossa matriz de embeddings V para X. Dessa forma, podemos definir as matrizes das seguintes formas:

- **Queries**: $Q = XM_Q$
- **Keys**: $K = XM_K$
- **Values**: $V = XM_V$

Aplicando essas matrizes de peso, conseguimos fazer com que nosso modelo possa aprender o que é mais importante para o contexto da palavra, e não apenas utilizar do embedding da palavra como estava sendo feito anteriormente. A partir disso, podemos reescrever a equação de atenção como:

$$\begin{aligned} S &= QK^T = XM_Q M_K^T X^T \\ W &= \text{softmax}\left(\frac{S}{\sqrt{d}}\right) \\ Y &= WV = \text{softmax}\left(\frac{S}{\sqrt{d}}\right) XM_V \end{aligned} \tag{52}$$

Aqui, o fator $\frac{1}{\sqrt{d}}$ é utilizado para evitar que os valores do produto interno fiquem muito grandes, o que poderia prejudicar a normalização com softmax e o gradiente na hora de treinar o modelo.

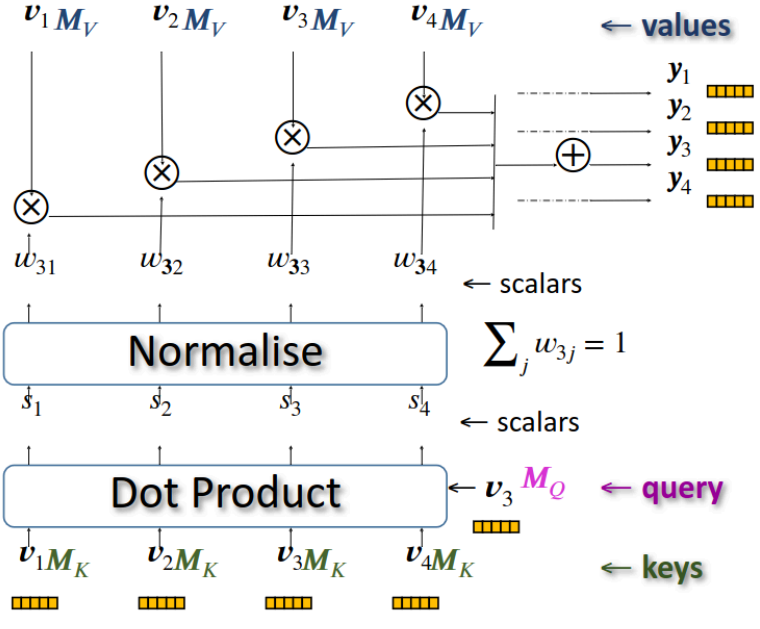


Figura 53: Ilustração do mecanismo de Self-Attention Aprendível

A ideia de duas matrizes de pesos diferentes, é que a matriz de **keys** e a matriz de **queries** podem aprender diferentes representações da mesma palavra, dependendo do contexto. Por exemplo, em uma frase, podemos ter a palavra *ferramenta* e a palavra *martelo*. A palavra *martelo* está **fortemente** ligada com a palavra ferramenta, mas a palavra *ferramenta* pode não está tão fortemente ligada, pois existem **vários tipos de ferramenta**. Ou seja, não temos uma relação de pesos **simétrica**, e é por isso que precisamos de duas matrizes de pesos **diferentes**, de forma que S não seja simétrica.

4.2.3 Multihead Self Attention

Quando estamos tratando de **linguagem natural**, é muito normal que, dentro de uma frase, a mesma palavra esteja associada de **formas diferentes** com a **mesma palavra**. Vamos pegar por

exemplo a frase *"She had already eaten the apple"*. Para se ter uma boa representação de **eaten**, temos que prestar atenção em **duas coisas**:

- A relação gramatical ligada ao tempo verbal *had* (junto com *eaten*, indica passado perfeito, uma ação que aconteceu antes de outra ação no passado);
- A relação semântica ligada ao objeto *apple* (o que foi comido);

Só que isso são dois padrões distintos que dão pesos diferentes à palavras diferentes. O que fazer então? Se lembra que em redes convolucionais, utilizávamos vários filtros para características diferentes? Um para narizes, outro para bocas, e assim vai? Aqui é a mesma ideia, vamos utilizar **mútiplas** "cabeças" de atenção, cada uma com seus próprios pesos aprendíveis. Suponha que tenhamos H cabeças de atenção indexadas de $h = 1, ..., H$ da seguinte forma:

$$H_h = \text{Attention}(Q_h, K_h, V_h) = \text{Softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}}\right) V_h \tag{53}$$

Aqui porém, nós geramos uma única matriz concatenada de todas as nossas cabeças de atenção, e aplicamos uma matriz de pesos aprendível M_O para gerar a saída final:

$$Y = \text{Concat}(H_1, ..., H_H) M_O \tag{54}$$

onde temos que $\text{Concat}(H_1, ..., H_H) \in \mathbb{R}^{N \times H \cdot d}$, $M_O \in \mathbb{R}^{H \cdot d \times d}$, logo, $Y \in \mathbb{R}^{N \times d}$

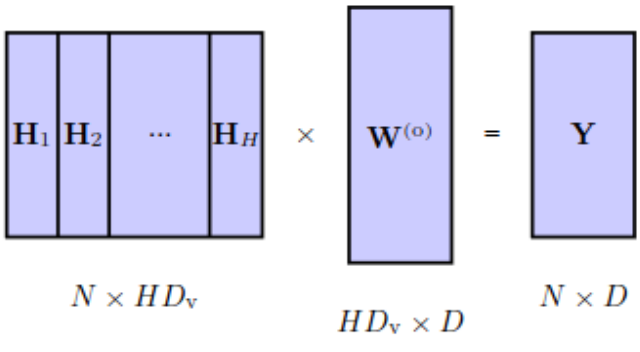


Figura 54: Concatenação para obter Y

4.3 Transformers

Introduzidos no famoso paper do Google *"Attention is All You Need"*, traz uma arquitetura de modelo desenvolvido para **tradução de textos**. A arquitetura é composta de dois blocos principais, o **encoder** e o **decoder**. O encoder é responsável por gerar uma representação vetorial do texto de entrada, enquanto o decoder é responsável por gerar a tradução do texto de entrada para o texto de saída. Na Figura 55, o bloco da esquerda é o **encoder** e o da direita é o **decoder**. Em ambos os blocos é utilizada das camadas de **multihead attention**. Dando um focono encoder, podemos ver algumas branches residuais, que, como já discutimos anteriormente, ajudam a evitar o problema de **vanishing gradient**. Algo interessante de se notar é que ao receber os embeddings do input, aplicamos nele o chamado **positional encoding**, que é uma forma de adicionar informação de posição para o modelo, pois, como já discutimos, a atenção é **contextual**, e a posição das palavras na frase é importante para o significado da frase. Fazemos algo como:

$$v_i' = v_i + p_i \tag{55}$$

onde v_i é o embedding da palavra i e p_i é o encoding de posição da palavra i. Existem várias formas de se definir p_i, pode ser uma função fixa, pode ser aprendido, ou até **ignorado**.

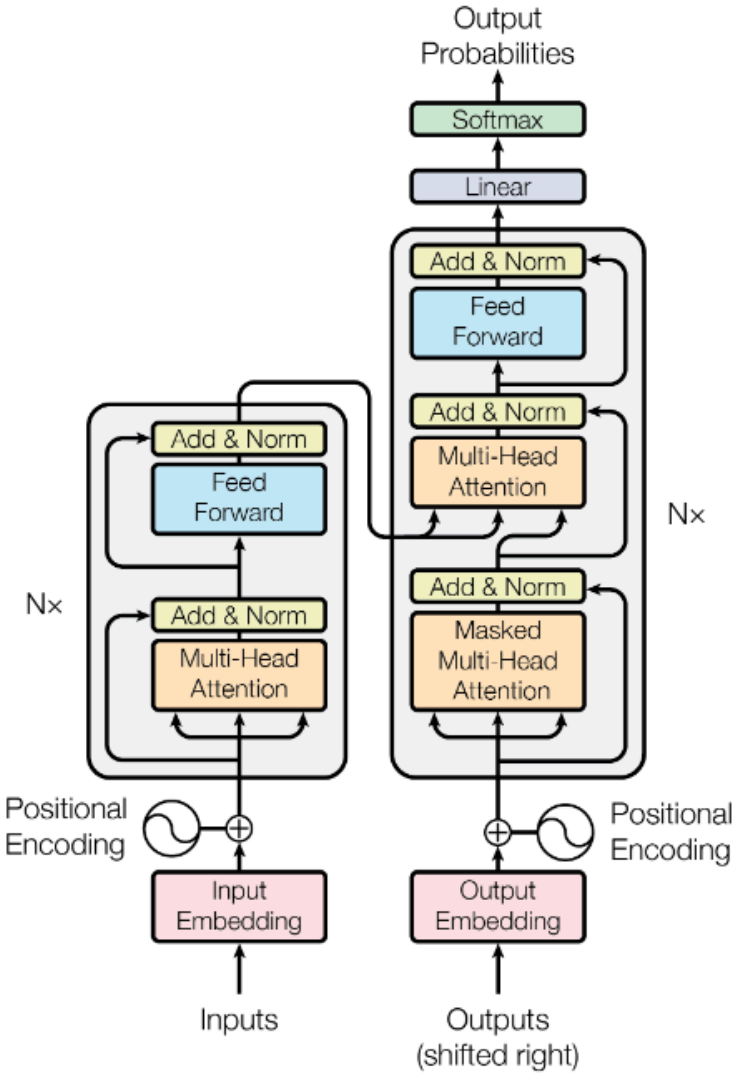


Figura 55: Arquitetura do Transformer

Já o decoder segue a mesma linha, no entanto, ele recebe os outputs anteriores e, no final, retorna as probabilidade de qual será a próxima palavra a ser adicionada na frase. Ele também utiliza de **multihead attention**, mas com uma diferença, ele utiliza de **masked attention**, que é uma forma de impedir que o modelo olhe para as palavras futuras na hora de gerar a próxima palavra. Isso é feito para evitar **vazamento de informação**.

4.4 Transformers vão substituir RNNs?

Podemos citar logo de cara algumas limitações que as RNNs possuem e que os Transformers não possuem:

- **Paralelização**: RNNs são sequenciais, ou seja, para processar a palavra i, precisamos processar todas as palavras anteriores. Já os Transformers podem processar todas as palavras de uma vez, o que permite uma paralelização muito maior.
- **Longo Alcance**: RNNs possuem dificuldade em capturar dependências de longo alcance, ou seja, quando a palavra i depende de uma palavra muito distante na frase. Já os transformers, dado poder computacional suficiente, teoricamente, tem uma janela de captura de dependência **infinita**, ou seja, a palavra i pode depender de qualquer palavra na frase, independente da distância.

Ainda podemos utilizar de RNNs em tarefas menores, mas para tarefas maiores e mais complexas, os Transformers trazem muitas vantagens em cima das RNNs.

4.5 Stand-Alone Self Attention

Introduzido no paper *"Stand-Alone Self-Attention in Vision Models"*, o Stand-Alone Self Attention (SASA) é um novo tipo de camada desenvolvida para **visão computacional**. Vimos em CNNs que, para capturar dependências espaciais que estão, de certa forma, distantes, utilizamos algumas gambiarras, como as atrous convolutions, que aumentam o tamanho do campo receptivo sem aumentar o número de parâmetros. No entanto, isso não é suficiente para capturar dependências espaciais mais complexas. O SASA utiliza de **multihead self attention** para capturar dependências espaciais de forma mais eficiente.

Vamos dar uma revisada rapidinho em como funciona a convolução em si. Dado um input (imagem) $V \in \mathbb{R}^{H \times W \times d_{\text{in}}}$ e um filtro $W \in \mathbb{R}^{k \times k \times d_{\text{in}}}$, o output $y_{ij} \in \mathbb{R}$ é dado por

$$y_{ij} = \sum_{|a-i| \leq \lfloor \frac{k}{2} \rfloor,\ |b-j| \leq \lfloor \frac{k}{2} \rfloor} W^T_{i-a,j-b} V_{ab} \tag{56}$$

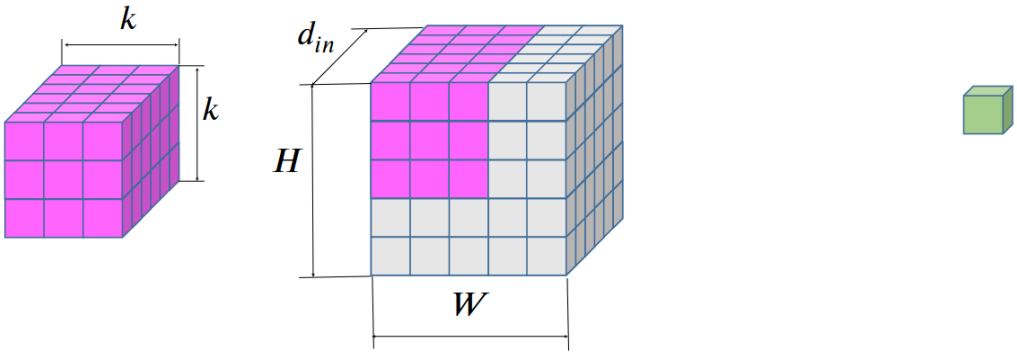


Figura 56: Ilustração da convolução

A convolução faz isso para todo i e j.

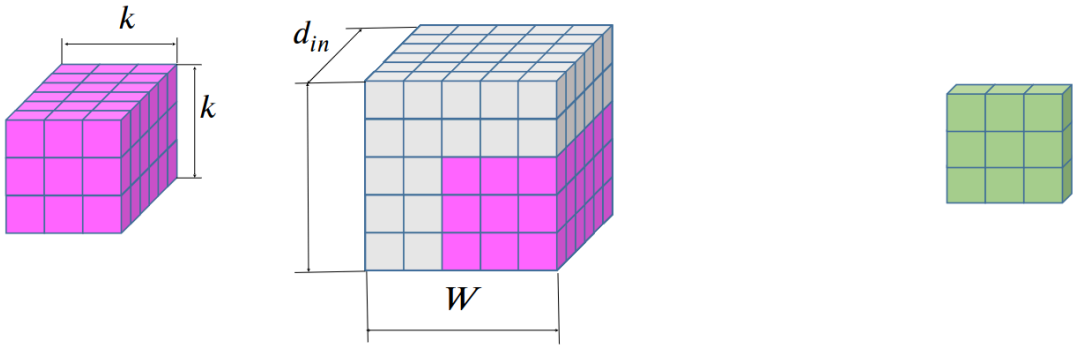


Figura 57: Ilustração da convolução

Em texto, todas as palavras podem se relacionar com as outras, mas aqui, nós vamos selecionar apenas uma região $k \times k$ para cada pixel $v_{ij} \in \mathbb{R}^{d_{\text{in}}}$. Chamamos essa região de **bloco de memória** e denotamos como $\mathcal{N}_k(i,j)$. Definimos então a saída $y_{ij} \in \mathbb{R}^{d_{\text{out}}}$ como:

$$\begin{aligned} y_{ij} &= \sum_{(a,b) \in \mathcal{N}_k(i,j)} \text{softmax}\big(Q_{ij}^T K_{ab}\big) V_{ab} \\ Q_{ij} &= W_Q^T v_{ij} \\ K_{ab} &= W_K^T v_{ab} \\ V_{ab} &= W_V^T v_{ab} \end{aligned} \tag{57}$$

onde $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$.

No entanto, na fórmula acima, a soma sobre a janela não sabe onde cada vizinho está. Se você embaralhar os pixels da janela, **o resultado é o mesmo**. Uma convolução não tem esse problema, porque cada posição da janela tem o seu próprio peso. Por isso é preciso informar a posição de alguma forma. A solução é então adicionar um termo $p_{a-i,b-j}$ que depende da posição relativa do pixel vizinho (a, b) em relação ao pixel central (i, j). A equação final fica então:

$$y_{ij} = \sum_{(a,b)\in\mathcal{N}_k(i,j)} \operatorname{softmax}\left[Q_{ij}^T\left(K_{ab} + p_{a-i,b-j}\right)\right] V_{ab} \tag{58}$$

Visual Foundation Models (Visual FM)